

i wanna do a recap on database sql and sql server tell me the points to recap

Absolutely. Since you already studied **DB / SQL / T-SQL** and you're preparing for **.NET backend interviews**, I'd structure the recap from fundamentals → SQL → SQL Server → advanced interview topics.

1. Database Fundamentals

- What is a database?
 - DBMS vs RDBMS
 - Tables, rows, columns
 - Primary Key
 - Foreign Key
 - Candidate Key / Alternate Key
 - Composite Key
 - Surrogate Key
 - Constraints
 - NOT NULL
 - UNIQUE
 - PRIMARY KEY
 - FOREIGN KEY
 - CHECK
 - DEFAULT
 - Relationships
 - One-to-One
 - One-to-Many
 - Many-to-Many
 - Normalization
 - 1NF
 - 2NF
 - 3NF
 - Denormalization
-

2. SQL Fundamentals

SELECT

- SELECT
- FROM
- WHERE
- DISTINCT
- TOP
- ORDER BY

Filtering

- Comparison operators
- AND, OR, NOT
- IN
- BETWEEN
- LIKE
- IS NULL
- IS NOT NULL

Aliases

- Column aliases
 - Table aliases
-

3. SQL Functions

Aggregate Functions

- COUNT
- SUM
- AVG
- MIN
- MAX

String Functions

- LEN
- LEFT
- RIGHT
- SUBSTRING

- CHARINDEX
- CONCAT
- LOWER
- UPPER
- TRIM

Date Functions

- GETDATE
- DATEADD
- DATEDIFF
- DATEPART
- YEAR
- MONTH
- DAY

NULL Handling

- ISNULL
 - COALESCE
 - Difference between NULL and empty string
-

4. GROUP BY & HAVING

Very important for interviews:

- GROUP BY
- HAVING
- Difference between WHERE and HAVING
- Aggregation with GROUP BY
- Multiple grouping columns

Example questions:

Find departments having more than 5 employees.

Find the average salary for each department.

5. JOINS

This is one of the most important sections.

Understand:

- INNER JOIN
- LEFT JOIN
- RIGHT JOIN
- FULL OUTER JOIN
- CROSS JOIN
- Self Join

And especially:

INNER vs LEFT JOIN

Understand what happens when there is **no matching record**.

Also practice joining 3–5 tables.

For example:

Customer

↓

Order

↓

OrderItem

↓

Product

6. Subqueries

- Scalar subquery
- Subquery with IN
- Subquery with EXISTS
- Correlated subquery
- EXISTS vs IN

Important interview question:

When would you use EXISTS instead of IN?

7. Set Operators

- UNION
- UNION ALL
- INTERSECT
- EXCEPT

Know the difference between:

UNION

and

UNION ALL

especially performance and duplicates.

8. SQL Commands / CRUD

DML

- INSERT
- UPDATE
- DELETE

DDL

- CREATE
- ALTER
- DROP
- TRUNCATE

DCL

- GRANT
- REVOKE
- DENY

TCL

- COMMIT
- ROLLBACK

- SAVEPOINT

Important:

DELETE vs TRUNCATE vs DROP

9. Views ★★

Understand:

- What is a View?
 - Why use Views?
 - Creating a View
 - Updating data through a View
 - View vs Table
 - View vs Stored Procedure
-

10. Stored Procedures ★★★

Very important for SQL Server interviews.

Recap:

- Creating Stored Procedures
- Parameters
- Input parameters
- Output parameters
- Return values
- Calling procedures
- IF
- CASE
- Variables
- Error handling
- Procedure vs Function

Also understand why stored procedures can be useful in some architectures.

11. User-Defined Functions

- Scalar-valued functions
 - Table-valued functions
 - Inline TVF
 - Multi-statement TVF
 - Function vs Stored Procedure
-

12. Transactions

Extremely important for backend development.

Understand:

```
BEGIN TRANSACTION  
COMMIT  
ROLLBACK
```

And **ACID**:

- Atomicity
- Consistency
- Isolation
- Durability

Also:

- Transaction lifecycle
 - Nested transactions
 - Error handling with transactions
-

13. Isolation Levels

Know:

- Read Uncommitted
- Read Committed
- Repeatable Read

- Serializable
- Snapshot

And the problems they solve:

- Dirty Read
- Non-Repeatable Read
- Phantom Read

This is a very common senior/intermediate SQL interview area.

14. Indexes

You specifically should spend time here.

Understand:

- What is an index?
- Why indexes improve reads
- Why indexes can hurt writes
- Clustered index
- Non-clustered index
- Composite index
- Unique index
- Covering index
- Included columns
- Index fragmentation
- Index maintenance

Most important:

Clustered vs Non-Clustered

Understand physically/logically what happens rather than just memorizing definitions.

15. Execution Plans

Learn the basics of SQL Server execution plans:

- Table Scan
- Index Scan
- Index Seek
- Key Lookup
- Sort
- Hash Match
- Nested Loops
- Merge Join

And understand:

Why is an Index Seek usually preferable to a Table Scan?

16. CTEs

Common Table Expressions:

WITH ...

Understand:

- Basic CTE
 - CTE with aggregation
 - CTE with joins
 - Recursive CTE
 - CTE vs Subquery
 - CTE vs Temporary Table
-

17. Window Functions

Very important for modern SQL interviews.

Learn:

- OVER()
- PARTITION BY
- ORDER BY

Functions:

- ROW_NUMBER()
- RANK()
- DENSE_RANK()
- NTILE()
- LAG()
- LEAD()

Example interview problem:

Find the second-highest salary in every department.

18. Temporary Tables

Understand:

- Local temporary table #Table
 - Global temporary table ##Table
 - Table variables
 - Temporary table vs table variable
 - When to use each
-

19. Identity & Sequences

SQL Server specific:

- IDENTITY
- IDENTITY_INSERT
- SCOPE_IDENTITY()
- @@IDENTITY
- IDENT_CURRENT
- SEQUENCE

Especially know why:

SCOPE_IDENTITY()

is generally safer than:

@@IDENTITY

for retrieving a newly generated identity value.

20. Triggers ★★

Since you previously wanted this for interview practice, definitely recap it.

- What is a Trigger?
 - DML triggers
 - AFTER triggers
 - INSTEAD OF triggers
 - INSERTED table
 - DELETED table
 - Trigger vs Stored Procedure
 - Advantages/disadvantages
 - Why excessive triggers can become problematic
-

21. Cursors

Know:

- What is a cursor?
- Cursor lifecycle
- DECLARE
- OPEN
- FETCH
- CLOSE
- DEALLOCATE

Most importantly:

Why should you generally prefer set-based operations over cursors?

22. SQL Server Architecture Basics

For a SQL Server-specific recap:

- SQL Server Database Engine
 - Database
 - Schema
 - Tables
 - Pages
 - Extents
 - Data files .mdf
 - Secondary data files .ndf
 - Log files .ldf
 - Transaction log
 - Buffer pool

You don't need DBA-level depth, but understand the concepts.

23. SQL Server Security

Know the difference between:

Login

User

Role

Permission

And:

- Authentication
 - Windows Authentication
 - SQL Server Authentication
- Authorization
- Server roles
- Database roles
- GRANT
- DENY
- REVOKE

24. Performance Optimization ★★

Very important for a .NET backend developer.

Recap:

- Proper indexes
- Avoid SELECT *
- Filtering early
- Avoid unnecessary joins
- SARGable queries
- Execution plans
- Index Seek vs Scan
- Avoid unnecessary subqueries
- Pagination
- Query optimization
- Statistics
- Parameterized queries

Also understand the classic problem:

```
WHERE YEAR(OrderDate) = 2026
```

vs a more index-friendly date range.

25. Database Design ★★

Practice designing databases from requirements.

For example:

E-commerce

Customer
Product
Category
Order

OrderItem
Payment

Think about:

- PKs
- FKs
- Cardinality
- Constraints
- Normalization
- Indexes
- Cascade behavior

This is particularly useful for your **.NET/EF Core** work.

26. SQL + EF Core Connection

Because you're targeting **.NET backend**, also recap how your SQL knowledge maps to EF Core:

- DbSet
- LINQ → SQL translation
- IQueryable vs IEnumerable
- Tracking vs AsNoTracking
- Include
- Projection with Select
- JOIN
- GroupBy
- Pagination
- Transactions
- Concurrency
- Migrations
- Relationships
- Cascade delete
- Index configuration

And importantly:

Understand the SQL generated by EF Core rather than treating EF Core as a replacement for SQL knowledge.

Priority for Your Interview

If you're short on time, I'd prioritize like this:

Tier 1 — MUST KNOW

1. SELECT / WHERE / ORDER BY
2. JOINS
3. GROUP BY / HAVING
4. Aggregate functions
5. PK / FK / constraints
6. Normalization
7. Subqueries
8. CRUD
9. DELETE vs TRUNCATE vs DROP
10. Indexes
11. Transactions + ACID
12. Stored Procedures
13. Views

Tier 2 — VERY IMPORTANT

14. CTE
15. Window Functions
16. Isolation Levels
17. Execution Plans
18. Triggers
19. Functions
20. Temporary Tables
21. Query optimization
22. SQL Server architecture basics

Tier 3 — ADVANCED

23. Cursors

- 24. Recursive CTE
 - 25. Locking/blocking/deadlocks
 - 26. Advanced indexing
 - 27. Statistics
 - 28. Partitioning
 - 29. Query Store
 - 30. Backup/restore concepts
-

How I suggest we do the recap

Rather than reading all of this as theory, we can go **group by group**, exactly like your previous C# and Angular recap:

- Group 1:** Database fundamentals
- Group 2:** SELECT + filtering
- Group 3:** Functions + aggregation
- Group 4:** JOINS
- Group 5:** Subqueries + CTE
- Group 6:** Window functions
- Group 7:** CRUD + DDL/DML/TCL
- Group 8:** Keys + constraints + normalization
- Group 9:** Views + Stored Procedures + Functions
- Group 10:** Transactions + ACID + Isolation
- Group 11:** Indexes + Execution Plans
- Group 12:** Triggers + Cursors
- Group 13:** SQL Server-specific topics
- Group 14:** Performance + optimization
- Group 15:** SQL interview problems

For each group, I can give you **the important interview questions first, then concise answers, then practical SQL exercises.**

ok let's go

Group 1 — Database Fundamentals

Let's start with the foundation. Focus on understanding the concepts, not memorizing definitions.

1. What is a Database?

A **database** is an organized collection of data that allows us to store, retrieve, update, and manage information efficiently.

2. What is DBMS?

DBMS (Database Management System) is software used to create and manage databases.

Examples:

- SQL Server
- MySQL
- PostgreSQL
- Oracle

3. DBMS vs RDBMS

DBMS

- General database management system.
- Doesn't necessarily use relationships between tables.

RDBMS

- Stores data in **tables**.
- Tables can have relationships.
- Usually follows relational principles.

SQL Server is an **RDBMS**.

4. What is a Table?

A table stores data in:

Columns → define the properties
Rows → represent individual records

Example:

Id	Name	Salary
1	Ahmed	10000
2	Khaled	12000

Id, Name, and Salary → columns

Each employee → row/record

5. Primary Key ★

A **Primary Key** uniquely identifies every row in a table.

Properties:

- Must be unique.
- Cannot be NULL.
- A table has one primary key constraint.
- It can consist of multiple columns → **composite primary key**.

Example:

Employee

Id ← Primary Key

Name

Salary

6. Foreign Key ★

A **Foreign Key** creates a relationship between two tables.

Example:

Department

Id (PK)

Name

Employee

Id (PK)

Name

DepartmentId (FK)

Employee.DepartmentId references Department.Id.

This ensures **referential integrity**.

7. Primary Key vs Foreign Key

Primary Key	Foreign Key
Identifies a row	References another table
Unique	Can contain duplicates
Cannot be NULL	Can be NULL depending on relationship/design
Defines entity identity	Defines relationship

8. What is a Composite Key?

A key consisting of **more than one column**.

Common example:

OrderItem

OrderId

ProductId

Quantity

OrderId + ProductId could together form the primary key.

Individually they aren't unique, but together they are.

9. Constraints ★

Constraints enforce rules on your data.

Important SQL Server constraints:

- PRIMARY KEY
- FOREIGN KEY
- UNIQUE
- NOT NULL
- CHECK
- DEFAULT

Example:

Age CHECK (Age >= 18)

means invalid ages can't be inserted.

10. UNIQUE vs PRIMARY KEY

Both enforce uniqueness, but:

Primary Key

- Identifies the row.
- Only one primary key constraint per table.
- Cannot contain NULL.

UNIQUE

- Enforces uniqueness on a column/set of columns.
 - A table can have multiple UNIQUE constraints.
 - NULL handling differs in SQL Server and deserves attention.
-

11. Relationships ★★★

One-to-One

Person 1 — 1 Passport

One person has one passport.

One-to-Many

Department 1 — * Employees

One department has many employees.

This is extremely common.

Many-to-Many

Student * — * Course

You normally solve this using a **junction table**:

Student
↓
StudentCourse
↑
Course

12. Normalization ★★

Normalization organizes data to reduce:

- duplication
- inconsistency
- update problems

The important levels:

1NF

- Atomic values.
- No repeating groups.

2NF

- 1NF
- No partial dependency on part of a composite key.

3NF

- 2NF
- No transitive dependency.

For your interviews, you should be able to explain **why normalization is useful**, not just recite the definitions.

Interview Questions You Should Be Able to Answer

Try answering these yourself:

1. What is the difference between DBMS and RDBMS?
2. What is a primary key?
3. Can a table have two primary keys?
4. Can a primary key contain NULL?
5. What is a foreign key?
6. Can a foreign key contain duplicate values?
7. What is a composite key?
8. Primary Key vs Unique Constraint?
9. What are database constraints?
10. Explain one-to-one, one-to-many, and many-to-many.
11. How do you implement many-to-many relationships?
12. What is normalization?
13. Explain 1NF, 2NF, and 3NF.
14. Why would you sometimes intentionally denormalize a database?

Mini challenge

Imagine you're designing a hospital database with:

Doctor
Patient
Appointment

Tell me:

What would be the PK and FK in each table, and what relationships exist between them?

Then we'll move to **Group 2: SELECT + WHERE + ORDER BY + filtering**, with SQL interview questions and exercises.

ok let's move to the next group

Group 2 — SELECT, Filtering & Sorting

This group is basic, but interviewers use it to test whether you really understand SQL execution and filtering.

1. SELECT

Used to retrieve data.

```
SELECT Name, Salary  
FROM Employees;
```

Retrieve all columns:

```
SELECT *  
FROM Employees;
```

Interview question

Why should you generally avoid SELECT * in production?

Because it:

- Retrieves unnecessary columns.
 - Transfers more data.
 - Can increase I/O.
 - Makes queries less explicit.
 - Can cause problems if the table structure changes.
-

2. DISTINCT

Removes duplicate results.

```
SELECT DISTINCT DepartmentId  
FROM Employees;
```

If employees have:

```
DepartmentId  
1  
1
```

2
2
3

Result:

1
2
3

Important

DISTINCT applies to the **entire selected combination**.

```
SELECT DISTINCT DepartmentId, JobTitle
FROM Employees;
```

The combination of DepartmentId + JobTitle must be unique.

3. WHERE

Filters rows.

```
SELECT *
FROM Employees
WHERE Salary > 10000;
```

Only rows satisfying the condition are returned.

4. Comparison Operators

Know these:

=	equal
<>	not equal
!=	not equal
>	greater
<	less
>=	greater than or equal
<=	less than or equal

Example:

```
SELECT *  
FROM Employees  
WHERE Salary >= 10000;
```

5. AND / OR / NOT

AND

Both conditions must be true.

```
WHERE Salary > 10000  
AND DepartmentId = 2
```

OR

At least one condition must be true.

```
WHERE DepartmentId = 1  
OR DepartmentId = 2
```

NOT

Negates a condition.

```
WHERE NOT DepartmentId = 2
```

AND + OR precedence

This is an important interview trap.

Consider:

```
WHERE DepartmentId = 1  
  OR DepartmentId = 2  
  AND Salary > 10000
```

AND is evaluated before OR.

So it's effectively:

```
WHERE DepartmentId = 1  
OR (DepartmentId = 2 AND Salary > 10000)
```

If you mean:

Department 1 or 2, and salary > 10000

write:

```
WHERE (DepartmentId = 1 OR DepartmentId = 2)  
AND Salary > 10000
```

Use parentheses when the intended logic isn't obvious.

6. IN

Instead of:

```
WHERE DepartmentId = 1  
OR DepartmentId = 2  
OR DepartmentId = 3
```

you can write:

```
WHERE DepartmentId IN (1, 2, 3)
```

Very common.

7. NOT IN

```
WHERE DepartmentId NOT IN (1, 2, 3)
```

Important NULL issue

NOT IN can produce surprising results when the compared values involve NULL.

For example, if a subquery returns:

```
1  
2  
NULL
```

then NOT IN may cause no rows to qualify because of SQL's three-valued logic.
This becomes particularly important with subqueries.

8. BETWEEN

Used for ranges.

WHERE Salary **BETWEEN** 10000 **AND** 20000

BETWEEN is **inclusive**.

So:

10000 

15000 

20000 

9. LIKE

Used for pattern matching.

Starts with K

WHERE Name **LIKE** 'K%'

Ends with n

WHERE Name **LIKE** '%n'

Contains "al"

WHERE Name **LIKE** '%al%'

Exactly one unknown character

WHERE Name **LIKE** '_haled'

_ = one character

% = zero or more characters

10. NULL

This is very important.

NULL means:

The value is unknown / missing / not applicable.

It is **not**:

0
"
false

You cannot correctly test NULL using:

```
WHERE ManagerId = NULL
```

Instead:

```
WHERE ManagerId IS NULL
```

or:

```
WHERE ManagerId IS NOT NULL
```

11. ORDER BY

Sorts the result.

Ascending:

```
SELECT *  
FROM Employees  
ORDER BY Salary ASC;
```

Descending:

```
SELECT *  
FROM Employees  
ORDER BY Salary DESC;
```

ASC is the default.

So:

ORDER BY Salary

means:

ORDER BY Salary **ASC**

12. Multiple Columns in ORDER BY

You can sort using multiple columns:

```
SELECT *  
FROM Employees  
ORDER BY DepartmentId ASC, Salary DESC;
```

Meaning:

1. Sort by department ascending.
 2. Within each department, sort salary descending.
-

13. TOP

SQL Server-specific.

```
SELECT TOP 5 *  
FROM Employees  
ORDER BY Salary DESC;
```

Gets the top 5 highest-paid employees.

Why ORDER BY matters

Without:

ORDER BY

you should **not assume** which 5 rows SQL Server will return.

14. TOP WITH TIES

Interesting SQL Server feature:

```
SELECT TOP 3 WITH TIES *  
FROM Employees  
ORDER BY Salary DESC;
```

Suppose salaries are:

```
50000  
40000  
30000  
30000  
20000
```

TOP 3 WITH TIES can return:

```
50000  
40000  
30000  
30000
```

because both employees tied at the third position.

15. OFFSET / FETCH

Useful for pagination.

```
SELECT *  
FROM Employees  
ORDER BY Id  
OFFSET 20 ROWS  
FETCH NEXT 10 ROWS ONLY;
```

Meaning:

Skip the first 20 rows and return the next 10.

This is important for backend APIs.

For example:

Logical Query Processing Order

This is **very important** for SQL interviews.

When you write:

```
SELECT ...  
FROM ...  
WHERE ...  
GROUP BY ...  
HAVING ...  
ORDER BY ...
```

SQL logically processes it approximately as:

```
FROM  
↓  
WHERE  
↓  
GROUP BY  
↓  
HAVING  
↓  
SELECT  
↓  
DISTINCT  
↓  
ORDER BY  
↓  
TOP / OFFSET-FETCH
```

This explains many SQL behaviors.

For example, why can't you normally use a SELECT alias inside WHERE?

Because logically:

```
WHERE
```

happens **before**

SELECT

Interview Questions

Make sure you can answer these:

Q1

What's the difference between WHERE and HAVING?

We'll cover HAVING deeply in the aggregation group, but the short answer:

- WHERE filters **rows before grouping**
 - HAVING filters **groups after grouping**
-

Q2

What's the difference between:

= NULL

and:

IS NULL

Answer: NULL represents an unknown value, so normal equality comparison doesn't work. Use IS NULL.

Q3

What's the difference between IN and BETWEEN?

IN checks whether a value matches one of several specified values.

BETWEEN checks whether a value falls within an inclusive range.

Q4

What's the difference between % and _ in LIKE?

% → zero or more characters

_ → exactly one character

Q5

What's the difference between DISTINCT and GROUP BY?

They can sometimes produce similar results, but they serve different purposes:

DISTINCT → remove duplicate result combinations.

GROUP BY → create groups, usually for aggregation.

Practice

Using:

Employees

Id

Name

Age

Salary

DepartmentId

HireDate

Try these without looking anything up:

1. Get all employees whose salary is greater than 15,000.
2. Get employees between 25 and 35 years old.
3. Get employees from departments 1, 3, or 5.
4. Get employees whose names start with A.

5. Get employees whose salary is between 10,000 and 20,000, sorted from highest salary to lowest.
6. Get the 5 highest-paid employees.
7. Get distinct department IDs.
8. Get employees who don't have a department.
9. Get the second page of employees assuming 10 employees per page.

After this, **Group 3 will be Aggregate Functions + GROUP BY + HAVING**, which is where SQL starts becoming much more interesting.

go

Group 3 — Aggregate Functions, GROUP BY & HAVING

This is one of the **most important SQL interview groups**. You should be comfortable writing these queries without thinking too much.

1. Aggregate Functions

Aggregate functions take **multiple rows** and produce a result.

The main ones:

Function	Purpose
COUNT()	Count rows/values
SUM()	Calculate total
AVG()	Calculate average
MIN()	Find minimum
MAX()	Find maximum

Assume:

Employees

Id	Name	Salary	DeptId
1	Ahmed	10000	1
2	Khaled	15000	1
3	Ali	20000	2
4	Omar	12000	2
5	Sara	18000	2

2. COUNT()

Count all rows

```
SELECT COUNT(*)  
FROM Employees;
```

Result:

5

COUNT(column)

```
SELECT COUNT(DepartmentId)  
FROM Employees;
```

Important:

COUNT(column) does **not count NULL values**.

While:

```
COUNT(*)
```

counts rows.

Very common interview question

What's the difference between:

```
COUNT(*)
```

and

```
COUNT(ColumnName)
```

Answer:

- COUNT(*) → counts rows.
 - COUNT(ColumnName) → counts non-NULL values in that column.
-

3. SUM()

Calculate the total.

```
SELECT SUM(Salary)
FROM Employees;
```

Result:

```
75000
```

You can combine it with WHERE:

```
SELECT SUM(Salary)
FROM Employees
WHERE DepartmentId = 1;
```

4. AVG()

```
SELECT AVG(Salary)
FROM Employees;
```

Important:

AVG() ignores NULL values.

For example:

```
10000
20000
NULL
```

Average is:

```
15000
```

not:

10000

5. MIN() and MAX()

```
SELECT MIN(Salary)
FROM Employees;
```

```
SELECT MAX(Salary)
FROM Employees;
```

You can retrieve multiple aggregates together:

```
SELECT
  COUNT(*) AS EmployeeCount,
  SUM(Salary) AS TotalSalary,
  AVG(Salary) AS AverageSalary,
  MIN(Salary) AS MinimumSalary,
  MAX(Salary) AS MaximumSalary
FROM Employees;
```

6. GROUP BY

This is the key concept.

Without GROUP BY:

```
SELECT AVG(Salary)
FROM Employees;
```

You get **one average for the entire table**.

But suppose you want:

Average salary for each department.

Use:

```
SELECT DepartmentId, AVG(Salary)
FROM Employees
GROUP BY DepartmentId;
```

Conceptually:

Employees
↓
GROUP BY DepartmentId
↓
Department 1 → employees
Department 2 → employees
Department 3 → employees
↓
AVG(Salary) for each group

7. GROUP BY Multiple Columns

You can group by more than one column:

```
SELECT DepartmentId, JobTitle, COUNT(*)  
FROM Employees  
GROUP BY DepartmentId, JobTitle;
```

SQL creates groups based on the **combination**:

DepartmentId + JobTitle

8. Important GROUP BY Rule

If you use GROUP BY, every selected column must generally be either:

1. In the GROUP BY, or
2. Inside an aggregate function.

For example:

```
SELECT DepartmentId, Name, AVG(Salary)  
FROM Employees  
GROUP BY DepartmentId;
```

 Invalid.

Why?

Because SQL doesn't know **which Name** it should return for a department containing multiple employees.

Correct:

```
SELECT DepartmentId, AVG(Salary)
FROM Employees
GROUP BY DepartmentId;
```

Or:

```
SELECT DepartmentId, Name, Salary
FROM Employees
GROUP BY DepartmentId, Name, Salary;
```

9. WHERE + GROUP BY

You can filter rows **before** grouping.

Example:

Calculate the average salary per department, but only consider employees earning more than 10,000.

```
SELECT DepartmentId, AVG(Salary)
FROM Employees
WHERE Salary > 10000
GROUP BY DepartmentId;
```

Conceptually:

All employees

↓

WHERE Salary > 10000

↓

Remaining employees

↓

GROUP BY DepartmentId

↓

AVG()

10. HAVING

HAVING filters **groups**.

Example:

Find departments with more than 5 employees.

```
SELECT DepartmentId, COUNT(*) AS EmployeeCount
FROM Employees
GROUP BY DepartmentId
HAVING COUNT(*) > 5;
```

This is something WHERE cannot directly do:

```
WHERE COUNT(*) > 5 -- ❌
```

because COUNT() is calculated after the rows are grouped.

11. WHERE vs HAVING

This is a **classic interview question**.

WHERE

Filters individual rows **before grouping**.

```
WHERE Salary > 10000
```

HAVING

Filters groups **after grouping**.

```
HAVING AVG(Salary) > 15000
```

Think:

WHERE

↓

Filter rows

↓

GROUP BY

↓

Create groups

↓
HAVING
↓
Filter groups

12. WHERE + HAVING Together

Example:

Find departments where employees earning more than 10,000 have an average salary greater than 15,000.

```
SELECT DepartmentId, AVG(Salary) AS AvgSalary
FROM Employees
WHERE Salary > 10000
GROUP BY DepartmentId
HAVING AVG(Salary) > 15000;
```

Execution conceptually:

```
All Employees
↓
WHERE Salary > 10000
↓
GROUP BY DepartmentId
↓
AVG(Salary)
↓
HAVING AVG(Salary) > 15000
↓
Result
```

13. GROUP BY with JOIN ★★ ★

This is where your real backend SQL starts.

Suppose:

Departments

Id
Name

Employees

Id
Name
Salary
DepartmentId

Question:

Get each department's name and average employee salary.

```
SELECT  
  d.Name,  
  AVG(e.Salary) AS AverageSalary  
FROM Departments d  
JOIN Employees e  
  ON d.Id = e.DepartmentId  
GROUP BY d.Name;
```

This pattern is **extremely common**.

14. COUNT + LEFT JOIN

This is a very useful interview concept.

Suppose you want:

Show all departments, including departments that have zero employees.

You need:

```
SELECT  
  d.Name,  
  COUNT(e.Id) AS EmployeeCount  
FROM Departments d  
LEFT JOIN Employees e
```

```
ON d.Id = e.DepartmentId  
GROUP BY d.Name;
```

Why COUNT(e.Id) instead of COUNT(*)?

Because with LEFT JOIN, a department with no employees still produces a row containing NULL employee columns.

COUNT(*) would count that joined row.

COUNT(e.Id) ignores the NULL.

This distinction is **very worth remembering**.

15. DISTINCT vs GROUP BY

These can sometimes produce similar results.

For example:

```
SELECT DISTINCT DepartmentId  
FROM Employees;
```

and:

```
SELECT DepartmentId  
FROM Employees  
GROUP BY DepartmentId;
```

can produce the same department list.

But their purposes are different:

DISTINCT > Remove duplicate result rows.

GROUP BY > Create groups, usually so you can perform aggregation.

16. COUNT(DISTINCT)

You can combine them:

```
SELECT COUNT(DISTINCT DepartmentId)
FROM Employees;
```

Meaning:

How many different departments have employees?

17. Aggregate Functions and NULL

Remember:

```
COUNT(*)
```

counts rows.

But:

```
COUNT(Column)
```

```
SUM(Column)
```

```
AVG(Column)
```

```
MIN(Column)
```

```
MAX(Column)
```

generally ignore NULL values.

Example:

```
Salary
```

```
---
```

```
10000
```

```
20000
```

```
NULL
```

```
AVG(Salary):
```

```
15000
```

not 10000.

Interview Questions

You should be able to answer these:

Q1

What's the difference between WHERE and HAVING?

Q2

Why can't you normally write:

```
WHERE COUNT(*) > 5
```

?

Q3

What's the difference between:

```
COUNT(*)
```

and:

```
COUNT(column)
```

?

Q4

What happens to NULL values when using AVG()?

Q5

Why does SQL require non-aggregated selected columns to appear in GROUP BY?

Q6

What's the difference between DISTINCT and GROUP BY?

Q7

Why might COUNT(e.Id) be preferable to COUNT(*) with a LEFT JOIN?



Practice Questions

Using:

Employees

Id

Name

Salary

DepartmentId

Departments

Id

Name

Try these:

1. Find the total salary of all employees.
2. Find the average salary of employees.
3. Find the highest salary in the company.
4. Find the number of employees in each department.
5. Find the average salary for each department.
6. Find departments that have more than 5 employees.
7. Find departments whose average salary is greater than 15,000.
8. Find the department with the highest average salary.
9. Display every department and the number of employees, including departments with zero employees.
10. 🔥 Find the highest salary in each department.

That last question is especially important because it leads directly into **subqueries, CTEs, and window functions**.

Next: **Group 4 — JOINS**, where we'll go deeply into INNER, LEFT, RIGHT, FULL, CROSS, and **SELF JOIN**, including the common interview traps.

go

Group 4 — JOINS ★★ ★

This is probably the **most important SQL topic** for a .NET backend interview. If you understand JOINS deeply, many SQL queries become much easier.

SQL Server uses JOINS to combine rows from multiple tables based on a relationship/condition. The main logical JOIN types are INNER, LEFT, RIGHT, FULL, and CROSS. ([Microsoft Learn](#))

Let's use these tables:

Departments

Id	Name
1	IT
2	HR
3	Sales
4	Finance

Employees

Id	Name	DepartmentId
1	Ahmed	1
2	Khaled	1
3	Sara	2
4	Omar	5

Notice:

- Department 3 has **no employees**
- Department 4 has **no employees**

- Omar has DepartmentId = 5, which doesn't exist in Departments — assuming the FK constraint isn't enforcing this for our example.
-

1. INNER JOIN ★★

Returns **only rows that have a match in both tables**. This is the default JOIN if you simply write JOIN. ([Microsoft Learn](#))

```
SELECT
    e.Name,
    d.Name AS DepartmentName
FROM Employees e
INNER JOIN Departments d
    ON e.DepartmentId = d.Id;
```

Result:

```
Ahmed  IT
Khaled IT
Sara   HR
```

Omar disappears because:

```
Employee.DepartmentId = 5
```

but there is no:

```
Department.Id = 5
```

Think of INNER JOIN as:

Give me only the matches.

2. LEFT JOIN ★★

A LEFT JOIN keeps **every row from the left table**, even if there is no matching row on the right. Unmatched right-side columns become NULL. ([Microsoft Learn](#))

```
SELECT
    d.Name AS DepartmentName,
    e.Name AS EmployeeName
FROM Departments d
```

```
LEFT JOIN Employees e
  ON d.Id = e.DepartmentId;
```

Result conceptually:

```
IT    Ahmed
IT    Khaled
HR     Sara
Sales  NULL
Finance NULL
```

Notice:

Sales and Finance are still there.

That's the whole point.

Think:

Give me everything from the LEFT table, and whatever matches from the RIGHT.

3. RIGHT JOIN

The opposite of LEFT JOIN.

```
SELECT
  d.Name AS DepartmentName,
  e.Name AS EmployeeName
FROM Departments d
RIGHT JOIN Employees e
  ON d.Id = e.DepartmentId;
```

It keeps **every employee**, even if their department doesn't match.

Result:

```
IT    Ahmed
IT    Khaled
HR     Sara
NULL  Omar
```

In practice, many developers prefer rewriting a RIGHT JOIN as a LEFT JOIN by switching the table order because it tends to be easier to read.

4. FULL OUTER JOIN

Keeps **everything from both tables**.

Matching rows are combined.

Unmatched rows from either side are kept with NULL on the other side.
([Microsoft Learn](#))

SELECT

```
d.Name AS DepartmentName,  
e.Name AS EmployeeName  
FROM Departments d  
FULL OUTER JOIN Employees e  
ON d.Id = e.DepartmentId;
```

Conceptually:

```
IT    Ahmed  
IT    Khaled  
HR    Sara  
Sales NULL  
Finance NULL  
NULL  Omar
```

So:

INNER → matching only

LEFT → everything from left

RIGHT → everything from right

FULL → everything from both

5. CROSS JOIN

This one is different.

It creates **every possible combination** of rows from both tables. SQL Server describes this as the Cartesian product. ([Microsoft Learn](#))

Suppose:

Departments = 4 rows
Employees = 4 rows

Then:

```
SELECT *  
FROM Departments  
CROSS JOIN Employees;
```

produces:

$4 \times 4 = 16$ rows

No ON condition.

For example:

IT	Ahmed
IT	Khaled
IT	Sara
IT	Omar
HR	Ahmed
HR	Khaled
HR	Sara
HR	Omar
...	

When is CROSS JOIN useful?

For things like:

Generate every combination of products and available sizes.

or:

Generate every combination of dates and stores.

But accidentally using it can create a **huge result set**.

6. SELF JOIN

A table can join to **itself**.

Imagine:

Employees

Id	Name	ManagerId
1	Ahmed	NULL
2	Khaled	1
3	Sara	1
4	Omar	2

You want:

Employee name + manager name.

You can join Employees to itself:

```
SELECT
  e.Name AS Employee,
  m.Name AS Manager
FROM Employees e
LEFT JOIN Employees m
  ON e.ManagerId = m.Id;
```

Result:

Ahmed	NULL
Khaled	Ahmed
Sara	Ahmed
Omar	Khaled

This is called a **self-join**.

The important thing is the aliases:

e → employee
m → manager

Even though both refer to the same table.

7. Joining More Than Two Tables ★★

Very important for backend work.

Suppose:

Customer
Order
OrderItem
Product

Relationships:

Customer
↓
Order
↓
OrderItem
↓
Product

You can do:

```
SELECT
  c.Name AS Customer,
  o.Id AS OrderId,
  p.Name AS Product
FROM Customer c
JOIN Orders o
  ON c.Id = o.CustomerId
JOIN OrderItems oi
  ON o.Id = oi.OrderId
JOIN Products p
  ON oi.ProductId = p.Id;
```

The key idea:

You don't need to join every table directly to every other table. You can use the relationships between them to reach the data you need.

8. JOIN + WHERE

Example:

Get employees who work in IT.

```
SELECT
  e.Name,
  d.Name AS Department
FROM Employees e
JOIN Departments d
  ON e.DepartmentId = d.Id
WHERE d.Name = 'IT';
```

Think:

```
JOIN
↓
Combine related rows
↓
WHERE
↓
Filter the result
```

9. The BIG LEFT JOIN Trap ★★ ★

This is one of the most important JOIN interview questions.

Suppose you want:

Show **all departments**, but only employees whose salary is greater than 10,000.

You might write:

```
SELECT
  d.Name,
  e.Name
FROM Departments d
LEFT JOIN Employees e
  ON d.Id = e.DepartmentId
WHERE e.Salary > 10000;
```

⚠ Be careful.

The WHERE condition eliminates rows where e.Salary is NULL.

That can effectively make your LEFT JOIN behave like an INNER JOIN for this condition.

Put the condition in ON

Instead:

```
SELECT
  d.Name,
  e.Name
FROM Departments d
LEFT JOIN Employees e
  ON d.Id = e.DepartmentId
  AND e.Salary > 10000;
```

Now the department is preserved even if it has no employee satisfying the salary condition.

This distinction matters because predicates in ON and WHERE can produce different results with outer joins. ([Microsoft Learn](#))

10. ON vs WHERE

This is worth memorizing conceptually.

ON

Defines:

How should the tables be matched?

```
ON e.DepartmentId = d.Id
```

WHERE

Defines:

Which rows from the resulting query should I keep?

```
WHERE e.Salary > 10000
```

For INNER JOIN, moving many conditions between ON and WHERE can produce the same result.

For OUTER JOIN, it can change the result. ([Microsoft Learn](#))

11. JOIN Does NOT Necessarily Mean Foreign Key

A common misconception:

“Can I JOIN only PK with FK?”

No.

The JOIN condition can compare other compatible columns too.

For example:

`ON e.Email = u.Email`

Although in well-designed relational databases, PK/FK relationships are a very common basis for joins.

SQL Server’s JOIN condition can use various logical predicates, not just equality. ([Microsoft Learn](#))

12. JOIN Can Produce Duplicate Rows ★★ ★

Suppose:

Department

IT

Employees

Ahmed IT

Khaled IT

If you join:

```
Department
JOIN Employees
```

you get:

```
IT  Ahmed
IT  Khaled
```

The department appears twice.

That's **not necessarily a problem**.

It happens because there are two matching employee rows.

This is especially important with **one-to-many relationships**.

13. JOIN + GROUP BY

Now combine what we learned in Group 3.

Find the number of employees in each department.

```
SELECT
    d.Name,
    COUNT(e.Id) AS EmployeeCount
FROM Departments d
LEFT JOIN Employees e
    ON d.Id = e.DepartmentId
GROUP BY d.Name;
```

This gives you:

```
IT      2
HR      1
Sales   0
Finance 0
```

Notice again:

```
COUNT(e.Id)
```

rather than:

`COUNT(*)`

because we want departments with no matching employees to have count 0.

14. JOIN vs UNION

Another common interview question.

JOIN

Combines **columns** from related tables.

Table A + Table B

↓

More columns

UNION

Combines **rows** from compatible result sets.

Query A

+

Query B

↓

More rows

So:

JOIN → horizontally

UNION → vertically

Not literally how the engine internally operates, but it's a useful mental model.

15. Physical JOIN Algorithms — SQL Server ★★

You don't need to deeply optimize these yet, but know the names.

SQL Server can execute joins using algorithms such as:

- Nested Loops
- Merge Join

- Hash Join
- Adaptive Join

The optimizer chooses an execution strategy based on things such as table sizes, indexes, and data distribution. ([Microsoft Learn](#))

We'll cover these properly later when we reach **Execution Plans & Performance**.

Interview Questions

Make sure you can answer these:

1. What is a JOIN?
 2. INNER JOIN vs LEFT JOIN?
 3. LEFT JOIN vs RIGHT JOIN?
 4. What does FULL OUTER JOIN return?
 5. What is CROSS JOIN?
 6. What is a SELF JOIN?
 7. Can you JOIN more than two tables?
 8. What's the difference between ON and WHERE?
 9. Why can a LEFT JOIN accidentally behave like an INNER JOIN?
 10. Why can a JOIN produce duplicate rows?
 11. JOIN vs UNION?
 12. What JOIN types does SQL Server support?
 13. What is a Cartesian product?
 14. What are Nested Loop, Hash, and Merge joins?
-

Practice — Do These Yourself

Using:

Customers

Id
Name

Orders

Id
CustomerId
OrderDate
TotalAmount

1.

Get every customer and their orders.

2.

Get only customers who have placed an order.

3.

Get all customers, including customers who have never placed an order.

4.

Find customers who have never placed an order.

5.

Get each customer's total number of orders.

6.

Get each customer's total order amount.

7. 🔥

Get the customer with the highest total order amount.

8. 🔥

Get all customers who have placed at least 3 orders.

9. 🔥

Suppose we add:

OrderItems

Id
OrderId
ProductId
Quantity

and:

Products

Id
Name
Price

Write the conceptual JOIN chain to get:

Customer → Order → OrderItem → Product

Don't worry about solving all of them perfectly yet. The important thing is that you can look at a database relationship diagram and immediately know **which JOIN you need and why**.

Next group: Subqueries + EXISTS + IN + correlated subqueries. This is where we'll start solving questions like *"find employees earning more than the company average"* and *"find the second-highest salary."*

Group 5 — Subqueries, IN, EXISTS & Correlated Subqueries ★★ ★

Subqueries are **queries inside another query**. In SQL Server, they can appear in SELECT, WHERE, HAVING, INSERT, UPDATE, DELETE, and even inside another subquery. ([Microsoft Learn](#))

We'll use:

Employees

Id

Name

Salary

DepartmentId

Departments

Id

Name

1. Basic Subquery

Suppose:

Find employees whose salary is greater than the average salary.

First, you could calculate:

```
SELECT AVG(Salary)
FROM Employees;
```

Then use that result:

```
SELECT *
FROM Employees
WHERE Salary > (
    SELECT AVG(Salary)
```

```
FROM Employees  
);
```

The inner query:

```
SELECT AVG(Salary)  
FROM Employees
```

returns **one value**.

The outer query compares every employee's salary with that value.

Mental model

Outer Query

↓

"Give me employees..."

↓

Inner Query

↓

"What's the average salary?"

↓

Outer Query continues

A scalar subquery used with >, <, =, etc. must return a single value; if it returns multiple rows, SQL Server raises an error. ([Microsoft Learn](#))

2. Subquery with IN

IN is useful when the subquery returns **multiple values**.

Example:

Get employees who work in departments located in Cairo.

Suppose:

Departments

Id	Name	City
1	IT	Cairo

2	HR	Giza
3	Sales	Cairo

Query:

```
SELECT *  
FROM Employees  
WHERE DepartmentId IN (  
    SELECT Id  
    FROM Departments  
    WHERE City = 'Cairo'  
);
```

The inner query might return:

1
3

Then conceptually:

```
WHERE DepartmentId IN (1, 3)
```

So employees from departments 1 and 3 are returned.

3. IN VS =

This is important.

=

Expects a **single value**:

```
WHERE DepartmentId = (  
    SELECT Id  
    FROM Departments  
    WHERE Name = 'IT'  
);
```

This works if the subquery returns exactly one row.

IN

Can handle multiple values:

```
WHERE DepartmentId IN (  
  SELECT Id  
  FROM Departments  
  WHERE City = 'Cairo'  
);
```

If the inner query returns:

```
1  
3  
5
```

IN is fine.

4. NOT IN

Opposite idea:

```
SELECT *  
FROM Employees  
WHERE DepartmentId NOT IN (  
  SELECT Id  
  FROM Departments  
  WHERE City = 'Cairo'  
);
```

Meaning:

Give me employees whose department isn't one of the Cairo departments.

NULL trap

Be careful when the subquery can return NULL.

Because SQL uses **three-valued logic** (TRUE, FALSE, UNKNOWN), NOT IN with a NULL in the subquery result can produce unexpected results.

This is one reason NOT EXISTS is often safer when checking non-existence.

5. EXISTS

EXISTS doesn't care about the actual values returned.

It asks:

Does the subquery return at least one row?

Example:

Get customers who have placed at least one order.

```
SELECT *  
FROM Customers c  
WHERE EXISTS (  
    SELECT *  
    FROM Orders o  
    WHERE o.CustomerId = c.Id  
);
```

The important part:

EXISTS (...)

returns logically:

TRUE

if at least one matching row exists, otherwise:

FALSE

SQL Server documents EXISTS specifically as an existence test. ([Microsoft Learn](#))

6. EXISTS Doesn't Care What You SELECT

These are effectively the same existence test:

```
WHERE EXISTS (  
    SELECT *  
    FROM Orders o  
    WHERE o.CustomerId = c.Id  
)
```

and:

```
WHERE EXISTS (  
  SELECT o.Id  
  FROM Orders o  
  WHERE o.CustomerId = c.Id  
)
```

and even:

```
WHERE EXISTS (  
  SELECT 1  
  FROM Orders o  
  WHERE o.CustomerId = c.Id  
)
```

Because you're asking:

Does at least one row exist?

You're not actually retrieving the selected value.

Microsoft's documentation notes that the select list of an EXISTS subquery doesn't determine the existence result. ([Microsoft Learn](#))

7. NOT EXISTS

Opposite:

Get customers who have **never** placed an order.

```
SELECT *  
FROM Customers c  
WHERE NOT EXISTS (  
  SELECT 1  
  FROM Orders o  
  WHERE o.CustomerId = c.Id  
)
```

Think:

EXISTS

→ at least one matching row

NOT EXISTS

→ zero matching rows

This is extremely common in real applications.

8. IN vs EXISTS ★★

Classic interview question.

IN

Conceptually:

Is this value inside the list returned by the subquery?

```
WHERE DepartmentId IN (  
    SELECT Id  
    FROM Departments  
)
```

EXISTS

Conceptually:

Does a matching row exist?

```
WHERE EXISTS (  
    SELECT 1  
    FROM Departments d  
    WHERE d.Id = e.DepartmentId  
)
```

A major difference is that EXISTS is naturally suited to **correlated existence checks**.

Don't memorize:

"EXISTS is always faster than IN."

That's too simplistic.

SQL Server can optimize semantically equivalent queries, and Microsoft notes that subqueries and equivalent joins can sometimes have the same execution plan. ([Microsoft Learn](#))

So choose based on **what you're expressing**, then inspect the execution plan when performance actually matters.

9. Correlated Subquery ★★ ★

This is the important one.

A correlated subquery references a column from the **outer query**.

Example:

Find employees whose salary is greater than the average salary of their own department.

```
SELECT *  
FROM Employees e  
WHERE e.Salary > (  
    SELECT AVG(e2.Salary)  
    FROM Employees e2  
    WHERE e2.DepartmentId = e.DepartmentId  
);
```

Look carefully:

```
WHERE e2.DepartmentId = e.DepartmentId
```

e.DepartmentId belongs to the **outer query**.

That's what makes this correlated.

Conceptually:

Employee Ahmed

↓

Find average salary of Ahmed's department

↓

Compare Ahmed's salary

Employee Khaled

↓

Find average salary of Khaled's department

↓

Compare Khaled's salary

...

The subquery depends on the current outer row. Microsoft describes correlated subqueries as subqueries that depend on values from the outer query and therefore conceptually repeat for the relevant outer rows. ([Microsoft Learn](#))

10. Normal vs Correlated Subquery

Non-correlated

```
SELECT *  
FROM Employees  
WHERE Salary > (  
    SELECT AVG(Salary)  
    FROM Employees  
);
```

The inner query doesn't reference the outer query.

Outer → Inner

Independent

Correlated

```
SELECT *  
FROM Employees e  
WHERE Salary > (  
    SELECT AVG(Salary)  
    FROM Employees e2  
    WHERE e2.DepartmentId = e.DepartmentId  
);
```

The inner query references:

e.DepartmentId

from the outer query.

Outer ↔ Inner

11. Subquery in SELECT

A subquery doesn't have to be in WHERE.

Example:

Display each employee and the company's average salary.

```
SELECT
    Name,
    Salary,
    (SELECT AVG(Salary) FROM Employees) AS CompanyAverage
FROM Employees;
```

Conceptually:

```
Ahmed 10000 15000
Khaled 20000 15000
Sara 15000 15000
```

SQL Server allows subqueries to be used where an expression can be used.
([Microsoft Learn](#))

12. Subquery vs JOIN ★★ ★

Many problems can be solved using either.

Subquery

```
SELECT *
FROM Employees
WHERE DepartmentId IN (
    SELECT Id
    FROM Departments
    WHERE Name = 'IT'
);
```

JOIN

```
SELECT e.*
FROM Employees e
JOIN Departments d
    ON e.DepartmentId = d.Id
WHERE d.Name = 'IT';
```

Both can return the same result.

Which is better?

Don't use a blanket rule.

Think about:

- Readability
- Whether you're checking existence
- Whether you need columns from the other table
- Actual execution plan
- Data size

SQL Server's optimizer can produce equivalent plans for semantically equivalent approaches. ([Microsoft Learn](#))

13. A Very Important Problem

Find the second-highest salary

One way:

```
SELECT MAX(Salary)
FROM Employees
WHERE Salary < (
    SELECT MAX(Salary)
    FROM Employees
);
```

Think about it:

1. Find maximum salary

↓

2. Remove that salary

↓

3. Find maximum of what's left

↓

4. That's the second-highest salary

But be careful with **duplicate salaries** and with the distinction between:

second-highest **distinct** salary

and

salary of the second employee after sorting.

We'll revisit this when we reach **window functions**, where `DENSE_RANK()` gives a cleaner solution for many ranking problems.

14. ANY / ALL — Know the Concept

These are less common in everyday backend code but can appear in interviews.

Suppose:

10
20
30

> ALL

> ALL (10, 20, 30)

means:

Greater than **every** value.

So effectively:

> 30

> ANY

means:

Greater than **at least one** value.

So:

> ANY (10, 20, 30)

effectively means:

> 10

SQL Server supports ANY, SOME, and ALL with comparison operators. ([Microsoft Learn](#))

For your current interview preparation, understand the concept but don't spend too much time here.

The 5 Things You MUST Remember

1. Scalar subquery

Returns **one value**:

```
WHERE Salary > (  
    SELECT AVG(Salary)  
    FROM Employees  
)
```

2. IN

Checks a value against a **list**:

```
WHERE DepartmentId IN (...)
```

3. EXISTS

Checks whether **at least one matching row exists**:

```
WHERE EXISTS (...)
```

4. NOT EXISTS

Checks that **no matching row exists**:

```
WHERE NOT EXISTS (...)
```

5. Correlated subquery

Inner query references the outer query:

WHERE e2.DepartmentId = e.DepartmentId

Interview Questions

Try to answer these:

1. What is a subquery?
 2. Where can a subquery be used?
 3. What's the difference between a scalar subquery and a multi-row subquery?
 4. IN vs EXISTS?
 5. NOT IN vs NOT EXISTS?
 6. What is a correlated subquery?
 7. How do you find employees earning more than the company average?
 8. How do you find employees earning more than their department's average?
 9. How do you find customers who have never placed an order?
 10. Can a subquery be replaced by a JOIN?
 11. What happens if a scalar subquery returns multiple rows?
 12. What do ANY and ALL mean?
-

Practice

Using:

Employees

Id

Name

Salary

DepartmentId

Departments

Id
Name

Customers

Id
Name

Orders

Id
CustomerId
TotalAmount

Try these:

1.

Find employees whose salary is greater than the **company average**.

2.

Find employees whose salary is greater than the **average salary of their department**.

3.

Find employees who work in the **IT department** using IN.

4.

Find employees who work in the **IT department** using EXISTS.

5.

Find customers who have placed at least one order using EXISTS.

6.

Find customers who have never placed an order using NOT EXISTS.

7. 🔥

Find the second-highest distinct salary.

8. 🔥

Find departments that have **at least one employee earning more than 20,000**.

9. 🔥

Find employees whose salary is greater than **every employee in department 2**.

10.

Explain whether this is correlated or non-correlated:

```
SELECT *  
FROM Employees e  
WHERE e.Salary > (  
    SELECT AVG(Salary)  
    FROM Employees  
);
```

Once you're comfortable with these, **Group 6 is CTEs + Recursive CTEs**, followed by **Window Functions**. Those two topics are where you'll start solving much more advanced SQL interview problems.

go

Group 6 — CTEs (Common Table Expressions)



CTEs are very useful for writing **complex SQL queries in a clean, readable way**. They are especially common in interview questions involving multiple steps, ranking, recursion, and hierarchical data.

A CTE is a named result set defined with WITH and available to the **single statement immediately following it**. It isn't automatically materialized as a temporary table. ([Microsoft Learn](#))

1. What is a CTE?

Instead of writing a complicated subquery:

```
SELECT *  
FROM (  
    SELECT DepartmentId, AVG(Salary) AS AvgSalary  
    FROM Employees  
    GROUP BY DepartmentId  
) x  
WHERE AvgSalary > 15000;
```

You can write:

```
WITH DepartmentSalary AS  
(  
    SELECT  
        DepartmentId,  
        AVG(Salary) AS AvgSalary  
    FROM Employees  
    GROUP BY DepartmentId  
)  
SELECT *  
FROM DepartmentSalary  
WHERE AvgSalary > 15000;
```

Much easier to read.

Think of it as:

```
WITH  
↓  
Create a temporary named result  
↓  
Use that result in the next statement
```

2. Basic CTE Syntax

The general structure is:

```
WITH CTE_Name AS  
(  
    SELECT ...  
)  
SELECT ...  
FROM CTE_Name;
```

Example:

```
WITH HighEarners AS  
(  
    SELECT *  
    FROM Employees  
    WHERE Salary > 20000  
)  
SELECT *  
FROM HighEarners;
```

The CTE itself isn't a permanent database object.

After the statement finishes, the CTE is gone.

3. CTE vs Temporary Table ★★

This is an important interview question.

CTE

```
WITH ...  
SELECT ...
```

- Exists for one statement.
- Mainly improves query organization/readability.
- Not automatically materialized.
- Useful for breaking complex queries into logical steps.

Temporary table

```
CREATE TABLE #Employees (...)
```

- Exists in the session/scope according to SQL Server's temp-table rules.
- Can be referenced by multiple statements.
- Can have indexes.
- Useful when you need to reuse an intermediate result.

Microsoft specifically notes that CTE results aren't materialized; if you need multiple references to the same result, a temporary object may be more appropriate. ([Microsoft Learn](#))

Interview answer:

A CTE is primarily a query expression used within one statement, while a temporary table stores an intermediate result that can be reused across statements.

4. CTE vs Subquery

Suppose:

Find departments whose average salary exceeds 15,000.

Subquery

```
SELECT *
FROM
(
    SELECT
        DepartmentId,
        AVG(Salary) AS AvgSalary
    FROM Employees
    GROUP BY DepartmentId
) d
WHERE AvgSalary > 15000;
```

CTE

```
WITH DepartmentSalary AS
(
    SELECT
        DepartmentId,
        AVG(Salary) AS AvgSalary
    FROM Employees
    GROUP BY DepartmentId
```

```
)  
SELECT *  
FROM DepartmentSalary  
WHERE AvgSalary > 15000;
```

Both can solve the same problem.

The CTE version is often easier to read when the intermediate query becomes complicated.

5. Multiple CTEs ★★

You can define multiple CTEs in the same WITH clause:

```
WITH  
DepartmentSalary AS  
(  
    SELECT  
        DepartmentId,  
        AVG(Salary) AS AvgSalary  
    FROM Employees  
    GROUP BY DepartmentId  
)  
HighSalaryDepartments AS  
(  
    SELECT *  
    FROM DepartmentSalary  
    WHERE AvgSalary > 15000  
)  
SELECT *  
FROM HighSalaryDepartments;
```

Notice:

```
DepartmentSalary  
    ↓  
HighSalaryDepartments  
    ↓  
Final SELECT
```

A later CTE can reference an earlier CTE in the same WITH clause. ([Microsoft Learn](#))

6. CTE with JOIN

Very common:

```
WITH EmployeeData AS
(
    SELECT
        e.Name AS EmployeeName,
        e.Salary,
        d.Name AS DepartmentName
    FROM Employees e
    JOIN Departments d
        ON e.DepartmentId = d.Id
)
SELECT *
FROM EmployeeData
WHERE Salary > 15000;
```

The CTE essentially gives you a clean intermediate dataset.

7. CTE with GROUP BY

Example:

Find departments with more than 5 employees.

```
WITH DepartmentCounts AS
(
    SELECT
        DepartmentId,
        COUNT(*) AS EmployeeCount
    FROM Employees
    GROUP BY DepartmentId
)
SELECT *
FROM DepartmentCounts
WHERE EmployeeCount > 5;
```

You could use HAVING directly, but the CTE becomes useful when you want to perform **additional processing on the aggregated result**.

8. CTE for Multi-Step Problems ★★ ★

This is where CTEs become really useful.

Question:

Find the department with the highest average salary.

First:

```
WITH DepartmentSalary AS
(
    SELECT
        DepartmentId,
        AVG(Salary) AS AvgSalary
    FROM Employees
    GROUP BY DepartmentId
)
SELECT TOP 1 *
FROM DepartmentSalary
ORDER BY AvgSalary DESC;
```

Logical process:

```
Employees
↓
GROUP BY Department
↓
Calculate average salary
↓
Name the result DepartmentSalary
↓
Sort by average salary
↓
TOP 1
```

9. CTE for Second Highest Salary

Another approach to a classic interview problem:

```
WITH Salaries AS
(
    SELECT DISTINCT Salary
    FROM Employees
)
SELECT MAX(Salary)
FROM Salaries
WHERE Salary < (
    SELECT MAX(Salary)
    FROM Salaries
);
```

But later we'll see that **window functions** make ranking problems much more elegant.

10. Recursive CTE ★★ ★

This is the advanced part.

A **recursive CTE** references **itself**.

It's useful for hierarchical data such as:

```
CEO
├── Manager A
│   ├── Employee 1
│   └── Employee 2
└── Manager B
    └── Employee 3
```

Imagine:

Employees

Id

Name
ManagerId

Example:

```
text id="xg2w5e" Id  Name      ManagerId 1  Ahmed   NULL 2   Khaled   1 3   Sara
1 4   Omar    2 5   Ali     2
```

You want to retrieve the entire hierarchy.

11. Structure of Recursive CTE

A recursive CTE has two important parts:

Anchor member

Gets the starting rows.

Recursive member

Gets the next level based on the previous result.

Conceptually:

```
Anchor
↓
Level 1
↓
Recursive step
↓
Level 2
↓
Recursive step
↓
Level 3
↓
...
```

SQL Server requires the recursive CTE to contain an anchor member and a recursive member. ([Microsoft Learn](#))

Example

WITH EmployeeHierarchy **AS**

```
(  
  -- Anchor  
  SELECT  
    Id,  
    Name,  
    ManagerId,  
    0 AS Level  
  FROM Employees  
  WHERE ManagerId IS NULL  
  
  UNION ALL  
  
  -- Recursive  
  SELECT  
    e.Id,  
    e.Name,  
    e.ManagerId,  
    h.Level + 1  
  FROM Employees e  
  JOIN EmployeeHierarchy h  
    ON e.ManagerId = h.Id  
)  
SELECT *  
FROM EmployeeHierarchy;
```

Result conceptually:

Id	Name	ManagerId	Level
1	Ahmed	NULL	0
2	Khaled	1	1
3	Sara	1	1
4	Omar	2	2
5	Ali	2	2

12. How Recursive CTE Executes

Suppose:

```
Ahmed
├── Khaled
│   ├── Omar
│   └── Ali
└── Sara
```

First:

```
Anchor
↓
Ahmed
```

Then:

```
Recursive step
↓
Khaled
Sara
```

Then:

```
Recursive step
↓
Omar
Ali
```

Then there are no more employees below them, so recursion stops.

This is essentially how SQL Server evaluates recursive CTEs: anchor results are produced first, then the recursive member repeatedly consumes the previous level until it returns no rows. ([Microsoft Learn](#))

13. MAXRECURSION ⚠

A badly designed recursive CTE can create an infinite loop.

SQL Server has MAXRECURSION to limit recursion depth.

Example:

```
WITH EmployeeHierarchy AS
(
    ...
)
```

```
SELECT *  
FROM EmployeeHierarchy  
OPTION (MAXRECURSION 10);
```

This limits the recursion to 10 levels.

SQL Server's default maximum recursion level for a statement is **100**; MAXRECURSION 0 removes the limit, so it should be used carefully. ([Microsoft Learn](#))

14. Recursive CTE Use Cases

You should remember these:

Organizational hierarchy

```
CEO  
↓  
Manager  
↓  
Employee
```

Categories

```
Electronics  
├── Phones  
│   ├── Android  
│   └── iPhone  
└── Laptops
```

File/folder structures

```
Root  
├── Folder A  
│   └── File  
└── Folder B
```

Bill of Materials

```
Car  
├── Engine  
│   ├── Piston  
│   └── Valve  
└── Wheel
```

15. CTE Important Restrictions

For a normal CTE, remember:

- It must be followed by one statement that uses it.
- It isn't a permanent object.
- It isn't automatically materialized.
- You can define multiple CTEs in one WITH.
- In SQL Server, if the preceding statement in the batch isn't terminated, starting with ;WITH is a common safe pattern. ([Microsoft Learn](#))

For recursive CTEs, SQL Server imposes additional restrictions on the recursive member, including restrictions around GROUP BY, HAVING, DISTINCT, and outer joins. ([Microsoft Learn](#))

Interview Questions

You should be able to answer:

1. What is a CTE?
2. Why would you use a CTE?
3. CTE vs subquery?
4. CTE vs temporary table?
5. Is a CTE a physical table?
6. Does a CTE store data permanently?
7. Can you have multiple CTEs?
8. Can one CTE reference another CTE?
9. What is a recursive CTE?
10. What are the anchor and recursive members?
11. What is MAXRECURSION?
12. Where would you use a recursive CTE?
13. Can a CTE be used with INSERT, UPDATE, or DELETE?
14. Does using a CTE automatically improve performance?

Important answer to #14:

No.

A CTE is primarily a query-organization mechanism. It doesn't automatically make a query faster. SQL Server can optimize the underlying query, and the CTE isn't automatically materialized. ([Microsoft Learn](#))

Practice

Using:

Employees

Id
Name
Salary
DepartmentId
ManagerId

Try:

1.

Create a CTE containing employees whose salary is greater than 15000, then select from it.

2.

Create a CTE calculating the average salary of each department, then return departments whose average is greater than 20000.

3.

Use two CTEs:

- First → calculate employee count per department.
- Second → keep departments with more than 5 employees.

4. 🔥

Use a CTE to find the department with the highest average salary.

5. 🔥

Use a recursive CTE to display the employee-management hierarchy.

6. 🔥

Explain the difference between:

CTE
Subquery
Temporary Table
View



One Important Correction to Your Mental Model

Don't think:

CTE = temporary table

That's the most common misunderstanding.

Think:

CTE = a named query expression that makes a complex statement easier to structure.

It has a temporary **scope**, but that doesn't mean SQL Server automatically creates a physical temporary table for it. ([Microsoft Learn](#))

Next: Group 7 — Window Functions ★★☆☆

This is a **very important interview group**:

- OVER()
- PARTITION BY
- ROW_NUMBER()
- RANK()
- DENSE_RANK()

- LAG()
- LEAD()
- Running totals
- Top-N per group
- Second/third highest salary
- Window functions vs GROUP BY

This is where you'll learn to solve many of the harder SQL interview questions cleanly.

go

Group 7 — Window Functions

This is **very important for SQL interviews**, especially ROW_NUMBER, RANK, DENSE_RANK, LAG, and LEAD.

1. What is a Window Function?

A **window function** calculates something across a set of related rows **without combining those rows into one row**.

The key syntax is:

```
FUNCTION(...) OVER (...)
```

For example:

```
SELECT
  Name,
  Salary,
  AVG(Salary) OVER () AS CompanyAverage
FROM Employees;
```

Every employee remains in the result, but each row also gets the company average.

That's the major difference from GROUP BY.

Microsoft describes OVER as defining the rows/partition on which the window function operates. ([Microsoft Learn](#))

2. OVER()

Think of:

OVER()

as:

“Calculate this function using the entire result set.”

Example:

```
SELECT
  Name,
  Salary,
  AVG(Salary) OVER () AS AvgSalary
FROM Employees;
```

If salaries are:

Name	Salary	AvgSalary
Ahmed	5000	7000
Ali	7000	7000
Omar	9000	7000

Unlike GROUP BY, **you didn't lose the individual employees.**

3. PARTITION BY

This is extremely important.

AVG(Salary) OVER (PARTITION BY DepartmentId)

means:

Calculate the average separately for each department.

Example:

```
SELECT
  Name,
  DepartmentId,
  Salary,
  AVG(Salary) OVER (
    PARTITION BY DepartmentId
  ) AS DepartmentAverage
FROM Employees;
```

Result conceptually:

Name	Department	Salary	DepartmentAverage
Ahmed	1	5000	6000
Ali	1	7000	6000
Omar	2	8000	9000
Hassan	2	10000	9000

Remember:

PARTITION BY $\neq$ GROUP BY

PARTITION BY **doesn't collapse rows.**

4. ORDER BY **inside** OVER

You can control the order in which the window function operates:

```
ROW_NUMBER() OVER (
  ORDER BY Salary DESC
)
```

This gives employees a ranking based on salary.

Important distinction:

```
ORDER BY Salary DESC
```

at the end of the query controls the **final result order**.

But:

OVER (ORDER BY Salary DESC)

controls the **window calculation**.

5. ROW_NUMBER()

Gives every row a **unique sequential number**.

```
SELECT
    Name,
    Salary,
    ROW_NUMBER() OVER (
        ORDER BY Salary DESC
    ) AS RowNum
FROM Employees;
```

Example:

Name	Salary	RowNum
Ahmed	10000	1
Ali	9000	2
Omar	9000	3
Hassan	7000	4

Even if salaries are equal, the row numbers are different.

ROW_NUMBER requires an ordering and can also restart numbering within each PARTITION BY. ([Microsoft Learn](#))

6. RANK()

RANK() gives the same rank to ties.

```
RANK() OVER (
    ORDER BY Salary DESC
)
```

Result:

Salary	RANK
10000	1
9000	2
9000	2
7000	4

Notice:

1, 2, 2, 4

There is a gap after the tie. ([Microsoft Learn](#))

7. DENSE_RANK()

Same idea, but **no gaps**.

```
DENSE_RANK() OVER (  
  ORDER BY Salary DESC  
)
```

Result:

Salary	DENSE_RANK
10000	1
9000	2
9000	2
7000	3

So:

ROW_NUMBER

1, 2, 3, 4

RANK

1, 2, 2, 4

DENSE_RANK

1, 2, 2, 3

This difference is a **very common interview question**. SQL Server provides all three as ranking functions. ([Microsoft Learn](#))

8. The Most Important Use: Top N per Group

Suppose you want:

Get the top 2 highest-paid employees from **each department**.

First:

```
ROW_NUMBER() OVER (  
    PARTITION BY DepartmentId  
    ORDER BY Salary DESC  
)
```

Then put that result into a CTE/subquery and filter:

```
WHERE RowNum <= 2
```

Conceptually:

```
Employees  
↓  
PARTITION BY Department  
↓  
ORDER BY Salary DESC  
↓  
ROW_NUMBER  
↓  
RowNum <= 2
```

This is one of the most useful real-world applications of window functions.

9. Second Highest Salary

Another classic interview question.

You can use:

```
DENSE_RANK() OVER (  
    ORDER BY Salary DESC  
)
```

Then:

```
WHERE Rank = 2
```

Why DENSE_RANK?

Suppose:

```
10000  
9000  
9000  
8000
```

The second-highest **distinct salary** is 9000.

DENSE_RANK gives:

```
10000 → 1  
9000 → 2  
9000 → 2  
8000 → 3
```

So both employees earning 9000 are returned.

10. LAG()

LAG() lets you access a value from a **previous row**.

Example:

```
SELECT  
    OrderDate,  
    Amount,  
    LAG(Amount) OVER (  
        ORDER BY OrderDate  
    ) AS PreviousAmount  
FROM Orders;
```

Result:

Date	Amount	PreviousAmount
Jan 1	100	NULL
Jan 2	150	100
Jan 3	120	150

Useful for:

- Comparing today's sales with yesterday's
 - Salary changes
 - Previous order
 - Previous transaction
 - Detecting changes
-

11. LEAD()

Opposite of LAG.

LEAD() gets a value from a **following row**.

```
LEAD(Amount) OVER (  
    ORDER BY OrderDate  
)
```

Result:

Date	Amount	NextAmount
Jan 1	100	150
Jan 2	150	120
Jan 3	120	NULL

So:

LAG → previous

LEAD → next

LEAD uses the ordering defined in its OVER clause to determine the following row.
([Microsoft Learn](#))

12. Running Total

Window functions are excellent for cumulative calculations.

```
SELECT
    OrderDate,
    Amount,
    SUM(Amount) OVER (
        ORDER BY OrderDate
        ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
    ) AS RunningTotal
FROM Orders;
```

Result:

Date	Amount	RunningTotal
Jan 1	100	100
Jan 2	200	300
Jan 3	150	450
Jan 4	300	750

Think:

```
100
100 + 200
100 + 200 + 150
100 + 200 + 150 + 300
```

ROWS/RANGE can define the exact window frame used by the calculation.
([Microsoft Learn](#))

13. Window Aggregate Functions

You can use normal aggregate functions as window functions:

```
SUM() OVER()
AVG() OVER()
COUNT() OVER()
```

MIN() OVER()
MAX() OVER()

For example:

```
SELECT
    Name,
    DepartmentId,
    Salary,
    COUNT(*) OVER (
        PARTITION BY DepartmentId
    ) AS EmployeesInDepartment
FROM Employees;
```

Every employee gets the number of employees in their department.

14. Window Function vs GROUP BY

GROUP BY

```
SELECT
    DepartmentId,
    AVG(Salary)
FROM Employees
GROUP BY DepartmentId;
```

Result:

	Department	Average
1		6000
2		9000

Employees disappeared into groups.

Window function

```
SELECT
    Name,
    DepartmentId,
    Salary,
    AVG(Salary) OVER (
        PARTITION BY DepartmentId
    )
FROM Employees;
```

Result:

```
Ahmed 1 5000 6000
Ali 1 7000 6000
Omar 2 8000 9000
Hassan 2 10000 9000
```

Interview answer:

GROUP BY combines rows into groups, while window functions calculate values across related rows while keeping the individual rows.

15. ROW_NUMBER for Removing Duplicates

Very common practical problem.

Suppose you have duplicate customers.

You can assign:

```
ROW_NUMBER() OVER (
  PARTITION BY Email
  ORDER BY Id
)
```

Then:

Email	RowNumber
a@gmail.com	1
a@gmail.com	2
a@gmail.com	3
b@gmail.com	1

You can keep RowNumber = 1 and identify/delete the duplicates.

★ Interview Questions You MUST Know

Q1. What is a window function?

A function that performs calculations across a set of related rows without collapsing them into a single row.

Q2. What does `OVER()` do?

Defines the window of rows on which the function operates.

Q3. What does `PARTITION BY` do?

Divides the result into groups/partitions, and the window calculation is performed separately for each partition.

Q4. Difference between `ROW_NUMBER`, `RANK`, and `DENSE_RANK`?

`ROW_NUMBER` → unique numbers

`RANK` → ties + gaps

`DENSE_RANK` → ties + no gaps

Q5. `LAG` vs `LEAD`?

`LAG` → previous row

`LEAD` → next row

Q6. Why use window functions instead of `GROUP BY`?

When you need aggregate/ranking information while still keeping every individual row.

Q7. How do you get top 3 employees from every department?

`ROW_NUMBER()` or `RANK()/DENSE_RANK()` with `PARTITION BY DepartmentId`, then filter the generated rank in an outer query/CTE.

Practice

Try these without looking at the answer:

1. Display every employee with their department's average salary.
2. Give every employee a salary rank within their department.
3. Find the top 3 highest-paid employees in each department.
4. Find the second-highest distinct salary.
5. Display each order with the previous order's amount.
6. Calculate a running total of sales ordered by date.
7. Find duplicate customers based on email using `ROW_NUMBER()`.

Once these are comfortable, **Group 8 is CRUD + DDL/DML/TCL**, which is more straightforward.

go

Group 8 — CRUD + DDL / DML / TCL

This group is mostly about **changing database structure, changing data, and controlling transactions**.

1. The Big Picture

Think of SQL commands in categories:

Category	Purpose	Examples
DDL	Define/change database structure	CREATE, ALTER, DROP, TRUNCATE
DML	Work with data	SELECT, INSERT, UPDATE, DELETE
TCL	Control transactions	COMMIT, ROLLBACK, SAVE TRANSACTION
DCL	Control permissions	GRANT, DENY, REVOKE

Microsoft classifies CREATE, ALTER, DROP, and TRUNCATE as DDL, while SELECT, INSERT, UPDATE, and DELETE are DML. ([Microsoft Learn](#))

2. CRUD

CRUD stands for:

C → Create
R → Read
U → Update
D → Delete

In SQL:

```
Create → INSERT  
Read  → SELECT  
Update → UPDATE  
Delete → DELETE
```

Example:

```
INSERT INTO Employees (Name, Salary)  
VALUES ('Ahmed', 5000);
```

```
SELECT *  
FROM Employees;
```

```
UPDATE Employees  
SET Salary = 6000  
WHERE Id = 1;
```

```
DELETE FROM Employees  
WHERE Id = 1;
```

3. INSERT

Insert one row:

```
INSERT INTO Employees (Name, Salary, DepartmentId)  
VALUES ('Ahmed', 5000, 2);
```

Multiple rows:

```
INSERT INTO Employees (Name, Salary)  
VALUES  
    ('Ahmed', 5000),  
    ('Ali', 6000),  
    ('Omar', 7000);
```

You can also insert from another query:

```
INSERT INTO EmployeesBackup (Name, Salary)  
SELECT Name, Salary  
FROM Employees;
```

Interview tip

Always prefer specifying the column names:

```
INSERT INTO Employees (Name, Salary)
VALUES (...);
```

rather than relying on the table's column order.

4. UPDATE

Basic:

```
UPDATE Employees
SET Salary = 7000
WHERE Id = 5;
```

Multiple columns:

```
UPDATE Employees
SET
    Salary = 7000,
    DepartmentId = 3
WHERE Id = 5;
```

Most important warning

If you forget WHERE:

```
UPDATE Employees
SET Salary = 7000;
```

you update **every employee**.

Same danger exists with DELETE.

5. DELETE

Delete specific rows:

```
DELETE FROM Employees  
WHERE Id = 5;
```

Delete all rows:

```
DELETE FROM Employees;
```

But the table itself still exists.

6. DELETE VS TRUNCATE VS DROP

🔥 **Very common interview question.**

DELETE

```
DELETE FROM Employees  
WHERE DepartmentId = 5;
```

- DML
 - Can use WHERE
 - Deletes selected rows
 - Table remains
 - Can be rolled back when executed within an appropriate transaction
-

TRUNCATE

```
TRUNCATE TABLE Employees;
```

- DDL in SQL Server
- Removes **all rows**
- Cannot use WHERE
- Table structure remains
- Resets the identity counter in the usual SQL Server case
- Typically much more efficient than deleting every row individually

Microsoft lists TRUNCATE TABLE under DDL. ([Microsoft Learn](#))

DROP

```
DROP TABLE Employees;
```

This removes the **table itself**.

So:

DELETE

↓

remove rows

TRUNCATE

↓

remove all rows

DROP

↓

remove table

Easy interview answer:

DELETE removes rows, TRUNCATE removes all rows while keeping the table structure, and DROP removes the table itself.

7. CREATE

Create a table:

```
CREATE TABLE Employees
(
  Id INT PRIMARY KEY,
  Name VARCHAR(100) NOT NULL,
  Salary DECIMAL(10,2),
  DepartmentId INT
);
```

You can create many other database objects:

```
CREATE DATABASE
CREATE TABLE
CREATE VIEW
CREATE PROCEDURE
CREATE FUNCTION
CREATE INDEX
...
```

8. ALTER

Used to modify an existing object.

Add a column:

```
ALTER TABLE Employees  
ADD Email VARCHAR(200);
```

Change a column definition:

```
ALTER TABLE Employees  
ALTER COLUMN Name VARCHAR(150) NOT NULL;
```

Add a constraint:

```
ALTER TABLE Employees  
ADD CONSTRAINT FK_Employees_Departments  
FOREIGN KEY (DepartmentId)  
REFERENCES Departments(Id);
```

Think:

CREATE → create something

ALTER → modify something

DROP → remove something

9. Transactions

Now we reach **TCL**.

A transaction is a group of operations that should be treated as **one unit of work**.

Example: transferring money:

```
Account A: -100  
Account B: +100
```

You don't want this:

A loses 100
B doesn't receive 100

Both operations should succeed, or neither should happen.

10. BEGIN TRANSACTION

Start a transaction:

BEGIN TRANSACTION;

Then perform operations:

UPDATE Accounts
SET Balance = Balance - 100
WHERE Id = 1;

UPDATE Accounts
SET Balance = Balance + 100
WHERE Id = 2;

If everything is successful:

COMMIT;

If something goes wrong:

ROLLBACK;

Conceptually:

BEGIN
↓
Operation 1
↓
Operation 2
↓
Success? — YES → COMMIT
|
NO
↓
ROLLBACK

11. COMMIT

COMMIT;

Means:

Permanently accept the changes made by the transaction.

12. ROLLBACK

ROLLBACK;

Means:

Undo the changes made since the transaction began, or since the relevant savepoint.

Example:

BEGIN TRANSACTION;

UPDATE Employees
SET Salary = Salary + 1000;

ROLLBACK;

The salary changes are undone.

13. SAVE TRANSACTION

You can create a **savepoint** inside a transaction.

BEGIN TRANSACTION;

UPDATE Employees
SET Salary = Salary + 1000;

SAVE TRANSACTION SalaryUpdate;

UPDATE Employees

```
SET DepartmentId = 5;
```

```
ROLLBACK TRANSACTION SalaryUpdate;
```

```
COMMIT;
```

The rollback goes back to the savepoint rather than cancelling the entire transaction.

Concept:

```
BEGIN
|
|— Salary update
|
|— SAVEPOINT
|
|— Department update
|
|— ROLLBACK TO SAVEPOINT
↓
Department update undone
Salary update remains
```

14. DCL — Permissions

You should at least recognize these:

```
GRANT
```

Give permission:

```
GRANT SELECT ON Employees TO User1;
```

```
DENY
```

Explicitly deny permission:

```
DENY DELETE ON Employees TO User1;
```

```
REVOKE
```

Remove a previously granted/denied permission:

REVOKE SELECT ON Employees **FROM** User1;

Don't spend too much time here for a junior .NET interview unless the job specifically involves database administration.

15. DELETE VS TRUNCATE — Interview Trap

A common simplistic answer is:

“TRUNCATE can't be rolled back.”

That's **not a good SQL Server interview answer**.

TRUNCATE TABLE can participate in a transaction and can be rolled back when used appropriately inside an explicit transaction.

For example:

BEGIN TRANSACTION;

TRUNCATE TABLE Employees;

ROLLBACK;

The important distinction is **not simply “DELETE can rollback, TRUNCATE can't.”**

Focus on:

	DELETE	TRUNCATE
WHERE	✓	✗
Remove selected rows	✓	✗
Remove all rows	✓	✓
Keep table	✓	✓
DML/DDL in SQL Server	DML	DDL
Usually faster for removing all rows	✗	✓

	DELETE	TRUNCATE
Identity reset	✗	✓ typically

16. DELETE and Foreign Keys

Suppose:

Departments

↓

Employees

An employee references a department.

You can't necessarily:

```
DELETE FROM Departments
WHERE Id = 1;
```

if employees still reference that department.

Depending on the FK configuration, SQL Server can prevent the deletion or cascade it.

This is where:

```
ON DELETE CASCADE
```

can become relevant.

★ Interview Questions

Q1. What is CRUD?

Create, Read, Update, Delete — represented by INSERT, SELECT, UPDATE, and DELETE.

Q2. DELETE vs TRUNCATE?

DELETE can remove selected rows using WHERE and is DML. TRUNCATE removes all rows, doesn't support WHERE, and is classified as DDL in SQL Server.

Q3. DELETE vs DROP?

DELETE removes rows; DROP removes the database object itself.

Q4. What happens if UPDATE has no WHERE?

It updates every row that the statement targets.

Q5. What is a transaction?

A group of operations treated as a single unit of work.

Q6. COMMIT vs ROLLBACK?

COMMIT makes the transaction's changes permanent; ROLLBACK undoes them.

Q7. What is a savepoint?

A point inside a transaction that allows you to roll back part of the transaction rather than the entire transaction.

Q8. What are DDL and DML?

DDL defines/modifies database structures; DML works with the data stored in those structures. ([Microsoft Learn](#))

One-Minute Memorization

DDL

CREATE

ALTER

DROP

TRUNCATE

DML

SELECT

INSERT

UPDATE

DELETE

TCL

BEGIN TRANSACTION

COMMIT

ROLLBACK

SAVE TRANSACTION

DCL
GRANT
DENY
REVOKE

Next: Group 9 — Keys, Constraints & Normalization, where we'll focus more deeply on **PK vs UNIQUE**, **composite keys**, **FK behavior**, **1NF/2NF/3NF**, and **normalization interview traps**.

go

Group 9 — Keys, Constraints & Normalization

This is another **high-priority interview group**.

1. Primary Key — PK

A **Primary Key** uniquely identifies each row.

```
CREATE TABLE Employees
(  
    Id INT PRIMARY KEY,  
    Name VARCHAR(100)  
);
```

Rules:

- Must be unique
- Cannot be NULL
- One PRIMARY KEY constraint per table
- Can consist of multiple columns
- SQL Server creates a unique index to enforce it. ([Microsoft Learn](#))

Example:

Id

1
2
3

You cannot have:

1
1

and you cannot have:

NULL

2. Foreign Key — FK

A Foreign Key establishes a relationship between tables and enforces **referential integrity**. ([Microsoft Learn](#))

Example:

Departments

Id
1
2
3

Employees

Id
Name
DepartmentId ← FK

Employees.DepartmentId references:

Departments.Id

Therefore:

```
INSERT INTO Employees
```

```
(  
    Name,  
    DepartmentId  
)
```

```
VALUES
```

```
(  
    'Ahmed',  
    99  
);
```

will fail if department 99 doesn't exist.

A foreign key can reference either a **primary key** or a **UNIQUE constraint** in the referenced table. ([Microsoft Learn](#))

3. PK vs FK

PK	FK
Identifies a row	References another row
Must be unique	Doesn't have to be unique
Cannot be NULL	Can be NULL unless NOT NULL
One PK constraint per table	Multiple FKs possible
Defines entity identity	Defines relationship

Example:

Department

Id ← PK

Employee

Id ← PK

DepartmentId ← FK

Many employees can have the same DepartmentId.

4. Composite Primary Key

A primary key can consist of multiple columns.

Example:

StudentCourse

StudentId

CourseId

You could define:

PRIMARY KEY (StudentId, CourseId)

This means:

StudentId + CourseId

must be unique **as a combination**.

So this is allowed:

StudentId | CourseId

-----|-----

1 | 10

1 | 20

2 | 10

But this isn't:

1 | 10

1 | 10 ← duplicate combination

Individual columns don't have to be unique. ([Microsoft Learn](#))

Typical use

Many-to-many relationships:

Students

↓

StudentCourses

↑

Courses

5. UNIQUE Constraint

UNIQUE means:

Values cannot be duplicated.

Example:

Email `VARCHAR(200) UNIQUE`

So:

ahmed@gmail.com
ahmed@gmail.com

isn't allowed.

But unlike a PK, a UNIQUE constraint can allow NULL in SQL Server. SQL Server's documented behavior allows one NULL value per unique-constrained column. ([Microsoft Learn](#))

Example

Employees

Id PK
Email UNIQUE
Phone UNIQUE

One table can have multiple UNIQUE constraints, while it has only one PK constraint.

6. PK vs UNIQUE

🔥 Very common interview question.

	PRIMARY KEY	UNIQUE
Uniqueness	✓	✓

	PRIMARY KEY	UNIQUE
NULL	✗	✓
Number per table	One	Multiple
Identifies entity	Usually yes	Not necessarily
Can be referenced by FK	✓	✓

SQL Server creates a unique index to enforce both types of uniqueness constraints. ([Microsoft Learn](#))

7. NOT NULL

Means:

This column must have a value.

Name **VARCHAR(100) NOT NULL**

This fails:

```
INSERT INTO Employees (Name)
VALUES (NULL);
```

8. DEFAULT

Provides a value automatically if one isn't supplied.

IsActive BIT **DEFAULT 1**

So:

```
INSERT INTO Employees (Name)
VALUES ('Ahmed');
```

can result in:

Name	IsActive
Ahmed	1

9. CHECK Constraint

Defines a rule for valid values.

Example:

```
Salary DECIMAL(10,2)  
CHECK (Salary >= 0)
```

This should fail:

```
Salary = -500
```

Another example:

```
Age INT CHECK (Age >= 18)
```

CHECK vs FK

CHECK:

"Is this value logically valid?"

FK:

"Does this referenced value exist in another table?"

Microsoft explicitly distinguishes them this way. ([Microsoft Learn](#))

10. Referential Integrity

Suppose:

```
Department 1  
  ↓  
Employee 5
```

If Employee 5 references Department 1, you shouldn't be able to delete Department 1 and leave an invalid reference.

The FK protects against this.

SQL Server can use different actions:

```
NO ACTION
CASCADE
SET NULL
SET DEFAULT
```

For example:

ON DELETE CASCADE

means deleting the parent can automatically delete the related child rows.
([Microsoft Learn](#))

11. One-to-One Relationship

Example:

User

Id

UserProfile

Id

UserId

One user has one profile.

You normally enforce the one-to-one relationship by making the FK unique.

Conceptually:

User.Id

↑

UserProfile.UserId

UNIQUE

Without uniqueness, multiple profiles could reference the same user.

12. One-to-Many

Most common relationship.

Department

1

|

|

N

Employees

One department:

DepartmentId = 1

can appear many times:

Employee 1 → Department 1

Employee 2 → Department 1

Employee 3 → Department 1

13. Many-to-Many

You normally need a junction table.

Example:

Students

|

|

StudentCourses

|

|

Courses

StudentCourses:

StudentId

CourseId

Often:

PRIMARY KEY (StudentId, CourseId)

14. Normalization

Normalization is about designing tables to:

Reduce unnecessary duplication and prevent data anomalies.

The most important levels for interviews:

1NF

2NF

3NF

15. First Normal Form — 1NF

A table is in **1NF** when each column contains atomic values and you don't store repeating groups in one field.

Bad:

Student

Name | PhoneNumbers

Ahmed | 010..., 011..., 012...

Better:

Student

Id

Name

StudentPhones

StudentId
Phone

Instead of:

Phone1
Phone2
Phone3

16. Second Normal Form — 2NF

2NF is mainly relevant when you have a **composite key**.

It requires:

The table is in 1NF and every non-key attribute depends on the **whole** composite key.

Example:

StudentCourse

StudentId
CourseId
StudentName
CourseName
Grade

Suppose:

PRIMARY KEY(StudentId, CourseId)

Problems:

StudentName → depends only on StudentId
CourseName → depends only on CourseId
Grade → depends on StudentId + CourseId

So StudentName and CourseName don't depend on the whole key.

Split them:

Students

StudentId
StudentName

Courses

CourseId
CourseName

StudentCourses

StudentId
CourseId
Grade

Now the data is properly separated.

17. Third Normal Form — 3NF

3NF deals with **transitive dependencies**.

Simple interview definition:

Every non-key column should depend on the key, the whole key, and nothing but the key.

Example:

Employee

EmployeeId
EmployeeName

DepartmentId
DepartmentName

Assume:

EmployeeId → DepartmentId
DepartmentId → DepartmentName

Therefore:

EmployeeId → DepartmentName

indirectly.

DepartmentName belongs to the Department table.

Better:

Employees

EmployeeId
EmployeeName
DepartmentId

Departments

DepartmentId
DepartmentName

★ 18. The Famous Normalization Formula

Remember this:

1NF

Atomic values

2NF

No partial dependency

3NF

No transitive dependency

Or:

1NF → atomic

2NF → whole key

3NF → nothing but the key

19. What Problems Does Normalization Prevent?

Three important anomalies:

Insert anomaly

You can't insert some information without unrelated information.

Update anomaly

The same information exists in multiple places, so you must update it everywhere.

Delete anomaly

Deleting one record accidentally removes information you still needed.

Example:

EmployeeId | Employee | Department | DepartmentPhone

If 50 employees belong to the same department, the department phone is duplicated 50 times.

Change the phone?

You need to update many rows.

Normalization separates:

Employees

Departments

20. Should We Always Normalize?

No.

In real systems, sometimes **denormalization** is intentionally used for:

- Performance
- Reporting
- Analytics
- Reducing expensive joins

So don't answer:

“Normalization is always better.”

Better:

“Normalization reduces redundancy and improves data integrity, but controlled denormalization can sometimes improve read performance.”

Interview Questions

Q1. Can a table have multiple primary keys?

No.

It can have only one PK constraint, but that PK can contain multiple columns.

Q2. Can a table have multiple UNIQUE constraints?

Yes.

For example:

```
Id    → PK
Email → UNIQUE
Username → UNIQUE
Phone → UNIQUE
```

Q3. Can FK be NULL?

Yes, unless the FK column is defined as NOT NULL.

Q4. Does FK automatically create an index?

No.

This is an important SQL Server detail.

A PK/UNIQUE constraint generates an index, but a foreign key does **not automatically** create one. An index on the FK can nevertheless be useful for joins and referential checks. ([Microsoft Learn](#))

Q5. Can FK reference something other than a PK?

Yes.

It can reference a UNIQUE constraint as well. ([Microsoft Learn](#))

Q6. What's the difference between 2NF and 3NF?

2NF removes partial dependencies on part of a composite key.

3NF removes dependencies where a non-key column depends on another non-key column.

Q7. What is a composite key?

A key consisting of multiple columns whose combination uniquely identifies a row.

Final Cheat Sheet

PRIMARY KEY

→ unique + NOT NULL

→ one per table

UNIQUE

- unique
- multiple allowed
- NULL allowed in SQL Server

FOREIGN KEY

- establishes relationship
- enforces referential integrity
- doesn't automatically create an index

CHECK

- validates a logical condition

DEFAULT

- supplies a default value

NOT NULL

- value is required

1NF → Atomic values

2NF → No partial dependency

3NF → No transitive dependency

Next: Group 10 — Views, Stored Procedures & Functions. This one is particularly important for **SQL Server/.NET backend interviews**.

go

Group 10 — Views, Stored Procedures & Functions

This is **very important for SQL Server + .NET backend interviews**.

Think of the three like this:

VIEW

- saved SELECT query

STORED PROCEDURE

- saved SQL program

FUNCTION

→ reusable calculation/query that returns a value or table

Microsoft describes a view as a stored SELECT, while a stored procedure can contain multiple T-SQL statements and parameters. ([Microsoft Learn](#))

1. Views

A **view** is a virtual table whose data comes from a query.

Example:

```
CREATE VIEW EmployeeDetails
AS
SELECT
    e.Id,
    e.Name,
    e.Salary,
    d.Name AS DepartmentName
FROM Employees e
JOIN Departments d
    ON e.DepartmentId = d.Id;
```

Then:

```
SELECT *
FROM EmployeeDetails;
```

You use it almost like a table.

A view normally stores the **query definition**, not a separate copy of the underlying data. ([Microsoft Learn](#))

2. Why use a View?

1. Simplify complex queries

Instead of repeatedly writing:

```
JOIN Employees
JOIN Departments
JOIN ...
```

you create a view once.

Then:

```
SELECT *
FROM EmployeeDetails;
```

2. Security

You can expose only certain columns/rows through a view instead of giving users direct access to the underlying tables. Microsoft specifically lists this as a use of views. ([Microsoft Learn](#))

For example, don't expose:

Employee

Id
Name
Salary
NationalId
PasswordHash

Create:

EmployeePublicView

Id
Name
Department

3. Do Views Accept Parameters?

Normal views don't accept parameters.

You cannot do:

```
GetEmployees(departmentId)
```

with a normal view.

If you need parameters, consider a:

- Stored procedure
- Table-valued function

Microsoft explicitly notes that views don't accept parameters. ([Microsoft Learn](#))

4. Can a View Have a JOIN?

Absolutely.

```
CREATE VIEW EmployeeDetails
AS
SELECT
    e.Name AS EmployeeName,
    d.Name AS DepartmentName
FROM Employees e
INNER JOIN Departments d
    ON e.DepartmentId = d.Id;
```

Views can be based on queries involving one or more tables.

5. Stored Procedures

A stored procedure is a named collection of T-SQL statements stored in the database.

Example:

```
CREATE PROCEDURE GetEmployees
AS
BEGIN
    SELECT *
```

```
FROM Employees;  
END;
```

Execute:

```
EXEC GetEmployees;
```

Stored procedures can accept input parameters and can have output parameters. They can also contain procedural logic such as IF and WHILE. ([Microsoft Learn](#))

6. Stored Procedure with Parameter

Example:

```
CREATE PROCEDURE GetEmployeesByDepartment  
    @DepartmentId INT  
AS  
BEGIN  
    SELECT *  
    FROM Employees  
    WHERE DepartmentId = @DepartmentId;  
END;
```

Execute:

```
EXEC GetEmployeesByDepartment  
    @DepartmentId = 2;
```

This is one major difference from a normal view.

7. Stored Procedure Can Modify Data

A procedure isn't restricted to SELECT.

It can perform:

```
INSERT  
UPDATE  
DELETE
```

For example:

```
CREATE PROCEDURE IncreaseSalary
```

```
    @EmployeeId INT,
```

```
    @Amount DECIMAL(10,2)
```

```
AS
```

```
BEGIN
```

```
    UPDATE Employees
```

```
    SET Salary = Salary + @Amount
```

```
    WHERE Id = @EmployeeId;
```

```
END;
```

Then:

```
EXEC IncreaseSalary 5, 1000;
```

8. Input vs Output Parameters

Input parameter

Pass data **into** the procedure:

```
@EmployeeId INT
```

Output parameter

Procedure can put a value **back into a variable** supplied by the caller.

Conceptually:

```
@EmployeeCount INT OUTPUT
```

Example:

```
CREATE PROCEDURE CountEmployees
```

```
    @Count INT OUTPUT
```

```
AS
```

```
BEGIN
```

```
    SELECT @Count = COUNT(*)
```

```
    FROM Employees;
```

```
END;
```

The caller can then retrieve the calculated value.

9. Stored Procedure vs View

🔥 Very common interview question.

View	Stored Procedure
Saved query	Saved program/SQL statements
Normally SELECT-based	Can contain SELECT/INSERT/UPDATE/DELETE
No parameters	Can accept parameters
Query with SELECT * FROM View	Execute using EXEC
Can be used in queries	Called as a procedure
Useful for abstraction/security	Useful for operations/business logic

Simple interview answer:

A view is mainly a reusable query that behaves like a virtual table, while a stored procedure is a reusable set of SQL statements that can accept parameters and perform operations.

10. Functions

A user-defined function (**UDF**) is a reusable routine that accepts parameters and returns either:

- A **single value**
- A **table**

Microsoft documents both scalar and table-valued functions. ([Microsoft Learn](#))

11. Scalar Function

Returns **one value**.

Example:

```
CREATE FUNCTION CalculateBonus
```

```
(  
    @Salary DECIMAL(10,2)  
)
```

```
RETURNS DECIMAL(10,2)
```

```
AS
```

```
BEGIN
```

```
    RETURN @Salary * 0.10;
```

```
END;
```

Use it:

```
SELECT
```

```
    Name,
```

```
    Salary,
```

```
    dbo.CalculateBonus(Salary) AS Bonus
```

```
FROM Employees;
```

Conceptually:

Salary

↓

Function

↓

One value

Example:

5000

↓

500

12. Table-Valued Function

Returns a **table**.

Example conceptually:

```
CREATE FUNCTION GetEmployeesByDepartment
```

```
(
```

```
    @DepartmentId INT
```

```

)
RETURNS TABLE
AS
RETURN
(
    SELECT *
    FROM Employees
    WHERE DepartmentId = @DepartmentId
);

```

Then:

```

SELECT *
FROM dbo.GetEmployeesByDepartment(2);

```

Think:

Scalar Function

↓

one value

Table-Valued Function

↓

multiple rows/columns

Microsoft explicitly describes table-valued functions as functions that return result sets usable in queries. ([Microsoft Learn](#))

13. Function vs Stored Procedure

🔥 **Very important.**

Function	Stored Procedure
Must return a value/table	Doesn't have to return a value
Can be used inside queries	Normally executed with EXEC
Can accept parameters	Can accept parameters
Scalar or table-valued	Can return result sets/output params
Generally intended for computation/querying	Can perform broader database operations

Function	Stored Procedure
Can be referenced in SELECT	Not normally embedded directly in a SELECT

For example:

Function:

```
SELECT dbo.CalculateBonus(Salary)
FROM Employees;
```

Procedure:

```
EXEC IncreaseSalary 5, 1000;
```

14. Can a Function Modify a Table?

For interview purposes, remember:

SQL Server UDFs have restrictions and aren't intended for arbitrary side-effecting data modifications.

So don't think:

Function = mini stored procedure

They're different concepts.

A function is primarily for **returning a value/table**, while a stored procedure can perform database operations such as modifying data. ([Microsoft Learn](#))

15. View vs Table

This is another common question.

Table

Stores actual data.

Employees

Id | Name | Salary

View

Stores the query definition.

EmployeeDetails

↓

SELECT ...
FROM Employees
JOIN Departments

When you query the view, SQL Server uses its underlying definition to produce the result.

So:

Table = data storage

View = saved query/interface over data

16. View vs CTE

You learned CTEs earlier.

CTE

WITH EmployeeData **AS** (...)
SELECT ...

Usually exists only for **one statement**.

View

CREATE VIEW EmployeeData
AS ...

The view is a **persistent database object** that you can query later.

So:

CTE
→ temporary query definition for one statement

VIEW

→ persistent database object

17. Stored Procedure vs C# Method

This is useful for a .NET interview.

They're conceptually similar:

C#

Method

↓

parameters

↓

logic

↓

result

and:

SQL

Stored Procedure

↓

parameters

↓

SQL logic

↓

result

But they execute in different environments:

C# method → application/server

Stored procedure → SQL Server

★ Interview Questions

Q1. What is a view?

A database object based on a SELECT query that presents data as a virtual table.

Q2. Does a view store data?

A normal view stores the query definition rather than a separate copy of the underlying data.

Q3. Can a view accept parameters?

No, a normal SQL Server view cannot accept parameters.

Q4. What is a stored procedure?

A named, reusable collection of T-SQL statements that can accept parameters and perform database operations.

Q5. What is a scalar function?

A function that returns a single value.

Q6. What is a table-valued function?

A function that returns a table/result set.

Q7. Function vs stored procedure?

A function returns a value/table and can be used in queries; a stored procedure is executed independently and can perform broader database operations.

Q8. View vs stored procedure?

A view is mainly a reusable query/virtual table and doesn't accept parameters; a stored procedure can accept parameters and contain multiple SQL operations.



Memorize This

VIEW

- virtual table
- saved SELECT
- no parameters

STORED PROCEDURE

- reusable SQL program
- parameters
- SELECT / INSERT / UPDATE / DELETE

→ EXEC

FUNCTION

→ reusable routine

→ returns something

SCALAR FUNCTION

→ one value

TABLE-VALUED FUNCTION

→ table

The easiest way to remember:

VIEW → "Give me this data."

PROCEDURE → "Do this operation."

FUNCTION → "Calculate/return this."

Next: Group 11 — Transactions, ACID & Isolation Levels. This is a **very high-value backend interview topic**, especially because it connects directly to Entity Framework and real .NET applications.

go

Group 11 — Transactions, ACID & Isolation Levels

This is **very important for .NET backend interviews** because transactions directly connect to EF Core, SaveChanges, concurrency, and database consistency.

1. What Is a Transaction?

A transaction is a **single unit of work**.

Example: bank transfer:

Account A: -100

Account B: +100

Both operations should succeed together.

If the second operation fails:

A → should NOT lose 100
B → should NOT receive 100

SQL Server describes a transaction as a single unit of work whose modifications are either committed or rolled back. ([Microsoft Learn](#))📞

2. ACID

🔥 **You MUST know this for interviews.**

ACID stands for:

A → Atomicity
C → Consistency
I → Isolation
D → Durability

A — Atomicity

All operations succeed, or all are rolled back.

Example:

Transfer \$100

1. Deduct \$100
2. Add \$100

If step 2 fails:

ROLLBACK

Step 1 is undone.

Remember:

All or nothing.

3. Consistency

A transaction moves the database from one valid state to another valid state.

Suppose:

Balance ≥ 0

is a business/database rule.

A transaction shouldn't leave the database violating that rule.

Consistency is maintained through things such as:

- Constraints
- Foreign keys
- CHECK constraints
- Data types
- Transaction rules
- Application/business rules

Remember:

Rules remain valid.

4. Isolation

Concurrent transactions should not improperly interfere with each other.

Suppose:

Transaction A → updating salary

Transaction B → reading salary

Isolation determines **what B is allowed to see while A is working.**

This is where **isolation levels** come in.

5. Durability

Once a transaction is successfully committed, its changes survive failures.

Example:

COMMIT

↓

Server crashes

↓

Database restarts

↓

Committed data remains

SQL Server's normal fully durable commit persists the relevant transaction log records before reporting the commit as successful. ([Microsoft Learn](#)) 📞

Remember:

Atomicity → all or nothing

Consistency → valid state

Isolation → concurrent transactions don't improperly interfere

Durability → committed data survives

6. Basic Transaction Syntax

BEGIN TRANSACTION;

UPDATE Accounts

SET Balance = Balance - 100

WHERE Id = 1;

UPDATE Accounts

SET Balance = Balance + 100

WHERE Id = 2;

COMMIT TRANSACTION;

If something goes wrong:

ROLLBACK TRANSACTION;

An explicit SQL Server transaction begins with BEGIN TRANSACTION and ends with COMMIT or ROLLBACK. ([Microsoft Learn](#))📞

7. Transaction Modes in SQL Server

You may hear about:

Autocommit

Each individual statement is automatically treated as its own transaction.

This is the normal default mode.

INSERT → transaction

UPDATE → transaction

DELETE → transaction

Explicit transaction

You manually control it:

BEGIN TRANSACTION;

...

COMMIT;

Implicit transaction

SQL Server automatically starts transactions, but you explicitly finish them with COMMIT or ROLLBACK.

For a junior interview, **autocommit vs explicit transaction** is usually enough.

SQL Server documents autocommit, explicit, implicit, and batch-scoped transaction modes. ([Microsoft Learn](#))📞

8. Isolation Levels

🔥 This is the most important part of this group.

SQL Server provides:

READ UNCOMMITTED
READ COMMITTED
REPEATABLE READ
SNAPSHOT
SERIALIZABLE

Isolation level controls locking/row-versioning behavior for statements in a connection. ([Microsoft Learn](#))📞

9. Dirty Read

Suppose Transaction A does:

Salary = 5000
↓
UPDATE to 7000

but hasn't committed.

Transaction B reads:

Salary = 7000

Then A performs:

ROLLBACK

Actual salary returns to:

5000

B read data that was never committed.

That's a:

Dirty Read

10. READ UNCOMMITTED

Lowest isolation level.

It allows dirty reads.

SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

Conceptually:

Transaction A

|
| uncommitted change
↓

Transaction B can read it

Problems:

Dirty reads ☒ possible

Non-repeatable ☒ possible

Phantom reads ☒ possible

Microsoft explicitly states that READ UNCOMMITTED allows reading rows modified by other transactions that haven't been committed. ([Microsoft Learn](#))📞

11. READ COMMITTED

This is the **default isolation level in SQL Server**. ([Microsoft Learn](#))📞

It prevents dirty reads.

Transaction A

↓
uncommitted update

Transaction B

↓
cannot read that uncommitted value

But it can still have:

Non-repeatable reads

Transaction B:

SELECT → 5000

Transaction A changes and commits:

5000 → 7000

Transaction B executes the same SELECT again:

SELECT → 7000

Same query, different value.

That's a **non-repeatable read**.

12. REPEATABLE READ

Guarantees that rows you've already read won't be changed by another transaction before your transaction finishes.

Example:

Transaction B:

SELECT salary → 5000

Transaction A:

tries to UPDATE salary

→ must wait

So repeated reads of the same row remain consistent.

But:

Phantom reads can still happen.

Microsoft notes that REPEATABLE READ prevents non-repeatable reads but still allows phantom reads. ([Microsoft Learn](#))📞

13. Phantom Read

Suppose Transaction B:

```
SELECT *  
FROM Employees  
WHERE Salary > 5000;
```

Result:

Ahmed
Ali

Then Transaction A inserts:

Omar → Salary 6000

and commits.

Transaction B runs the same query again:

Ahmed
Ali
Omar

A new row appeared.

That's a:

Phantom read

Think:

Non-repeatable read
→ existing row changed

Phantom read
→ new matching row appeared/disappeared

14. SERIALIZABLE

Highest standard isolation level.

It provides the strongest isolation by preventing other transactions from inserting/updating rows that would interfere with the transaction's range of data.

Example:

SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;

If you query:

WHERE Salary **BETWEEN 5000 AND 10000**

SQL Server can use range locks to prevent another transaction from inserting a qualifying row until the transaction completes. ([Microsoft Learn](#))☎

Advantage

Very strong consistency.

Disadvantage

Less concurrency and potentially more blocking.

So:

Don't use SERIALIZABLE everywhere.

Use it when the business operation actually requires that level of isolation.

15. SNAPSHOT

SNAPSHOT uses **row versioning** rather than making readers wait for writers in the same way as locking-based isolation.

A transaction sees a consistent version of the data from the point at which the transaction started.

Important properties:

Dirty read ✗
Non-repeatable ✗
Phantom ✗

Microsoft's isolation-level comparison shows SNAPSHOT prevents all three phenomena. ([Microsoft Learn](#))☎

16. The Table You MUST Memorize

Isolation	Dirty	Non-repeatable	Phantom
READ UNCOMMITTE	✓	✓	✓

Isolation	Dirty	Non-repeatable	Phantom
D			
READ COMMITTED	✗	✓	✓
REPEATABLE READ	✗	✗	✓
SNAPSHOT	✗	✗	✗
SERIALIZABLE	✗	✗	✗

This is directly aligned with Microsoft's SQL Server isolation-level summary.
[\(Microsoft Learn\)](#) 📞

17. Isolation Level Trade-off

Generally:

More isolation

↓

More consistency

↓

Less concurrency

↓

Potentially more blocking

Think:

READ UNCOMMITTED

↓

READ COMMITTED

↓

REPEATABLE READ

↓

SERIALIZABLE

SNAPSHOT is different because it uses row versioning rather than simply being “higher locking” than SERIALIZABLE.

18. Locks

Transactions can use locks to control concurrent access.

Two important types:

Shared Lock — S

Used for reading.

Multiple transactions can generally hold shared locks simultaneously.

Exclusive Lock — X

Used for modifications.

```
INSERT  
UPDATE  
DELETE
```

An exclusive lock prevents conflicting access while the modification is protected.

19. Blocking

Suppose:

```
Transaction A  
  UPDATE Employee  
  ↓  
holds lock
```

Transaction B:

```
UPDATE same Employee
```

B may have to **wait**.

That's:

Blocking

Blocking isn't automatically a bug.

Short-lived blocking is normal.

The problem is **long-running transactions and excessive blocking**.

Microsoft notes that transactions left open for long periods can prevent other sessions from accessing locked resources. ([Microsoft Learn](#))📞

20. Deadlock

🔥 Very common backend interview question.

A deadlock happens when two transactions are waiting for each other.

Example:

Transaction A
locks Employee 1
↓
waits for Employee 2

Transaction B
locks Employee 2
↓
waits for Employee 1

Visual:

A owns → 1
A wants → 2

B owns → 2
B wants → 1

Neither can continue.

SQL Server detects the deadlock and chooses one transaction as the **deadlock victim**, rolling it back so the other can continue.

21. How to Reduce Deadlocks

Common strategies:

- Keep transactions short
- Access resources in a consistent order
- Avoid unnecessary locks
- Use appropriate indexes
- Avoid unnecessary isolation levels
- Don't keep a transaction open while doing unrelated application work

For example, don't do:

```
BEGIN TRANSACTION
```

↓

```
DB update
```

↓

```
Call external API
```

↓

```
Wait 10 seconds
```

↓

```
More DB work
```

↓

```
COMMIT
```

That can hold resources much longer than necessary.

22. SQL Transactions vs EF Core

This is **very important for you as a .NET developer**.

When you call:

```
DbContext.SaveChanges()
```

EF Core can execute the changes within a transaction so that the related database modifications are treated atomically.

For example:

```
Add Patient
```

```
Add Consultation
```

```
Add Prescription
```

↓

```
SaveChanges()
```



transactional operation

If an error occurs during the operation, the changes can be rolled back rather than leaving a partially applied unit of work.

For more complex workflows, EF Core also supports explicit transaction APIs.

★ Interview Questions

Q1. What is ACID?

Atomicity, Consistency, Isolation, and Durability.

Q2. What is a dirty read?

Reading uncommitted data from another transaction.

Q3. What is a non-repeatable read?

Reading the same row twice and getting different values because another transaction modified and committed it between the reads.

Q4. What is a phantom read?

Repeating a query and finding additional or missing rows because another transaction inserted or deleted rows matching the query condition.

Q5. Default isolation level in SQL Server?

READ COMMITTED.

([Microsoft Learn](#))📖

Q6. Which isolation level allows dirty reads?

READ UNCOMMITTED.

Q7. Which prevents dirty + non-repeatable but allows phantom?

REPEATABLE READ.

Q8. Which provides the strongest standard locking isolation?

SERIALIZABLE.

Q9. What is a deadlock?

Two or more transactions wait for resources held by each other, so none can proceed.

Q10. Blocking vs deadlock?

Blocking: one transaction waits for another to release a resource.

Deadlock: transactions form a circular wait and cannot proceed without intervention.

One-Minute Cheat Sheet

ACID

A → All or nothing

C → Valid state

I → Transactions don't improperly interfere

D → Committed data survives

Dirty read

→ read uncommitted data

Non-repeatable read

→ same row, different value

Phantom read

→ same query, different set of rows

READ UNCOMMITTED

→ dirty reads possible

READ COMMITTED

→ default SQL Server

→ prevents dirty reads

REPEATABLE READ

→ prevents dirty + non-repeatable

→ phantom still possible

SNAPSHOT

→ row versioning

→ prevents dirty/non-repeatable/phantom

SERIALIZABLE

- strongest isolation
- range locking
- less concurrency

Next: Group 12 — Indexes & Execution Plans. This is one of the **highest-value SQL Server interview topics**, especially clustered vs nonclustered, composite indexes, included columns, seek vs scan, and SARGability.

ok

Group 12 — Indexes & Execution Plans

🔥 **Very high priority for SQL Server interviews.**

You should especially understand **clustered vs nonclustered, composite indexes, covering indexes, Index Seek vs Scan, and execution plans.**

1. What Is an Index?

An index is a database structure that helps SQL Server **find rows faster** instead of scanning the entire table.

Think about a book:

Without index:

Read every page → find "SQL"

With index:

Go to index → find page → jump directly there

Example:

```
CREATE INDEX IX_Employees_Name  
ON Employees(Name);
```

Now searching:

```
SELECT *  
FROM Employees  
WHERE Name = 'Ahmed';
```

can potentially use that index.

⚠ An index doesn't guarantee SQL Server will use it. The optimizer chooses a plan based on cost, statistics, available indexes, query shape, etc.

2. Why Not Index Every Column?

Because indexes aren't free.

An index:

- Takes disk/storage space
- Must be maintained when data changes
- Can make INSERT slower
- Can make UPDATE slower
- Can make DELETE slower
- Requires maintenance/statistics considerations

So:

Indexes improve reads but add write/storage/maintenance cost.

3. Clustered Index

A clustered index determines the **physical/logical ordering of the table's data pages according to the clustered key**.

Important SQL Server fact:

A table can have **only one clustered index**.

Why?

Because the table's rows can only have one clustered ordering.

Example:

```
CREATE CLUSTERED INDEX IX_Employees_Id  
ON Employees(Id);
```

If Id is the primary key, SQL Server commonly creates a clustered index for it by default unless you specify otherwise.

4. Nonclustered Index

A nonclustered index is a **separate index structure** that contains indexed key values and references to the underlying rows.

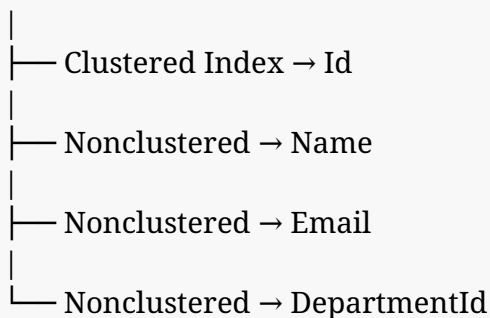
Example:

```
CREATE NONCLUSTERED INDEX IX_Employees_Name
ON Employees(Name);
```

A table can have **multiple nonclustered indexes**.

For example:

Employees



5. Clustered vs Nonclustered

🔥 Memorize this:

Clustered	Nonclustered
One per table	Multiple allowed
Organizes table data by key	Separate index structure
Data rows are at the leaf level	Leaf contains index keys + row locator
Often PK	Often search/filter/join columns

Microsoft documents that a table can have one clustered index and multiple nonclustered indexes. (learn.microsoft.com)📞

6. Primary Key ≠ Clustered Index

🔥 Important interview trap.

People sometimes say:

“Primary key is always clustered.”

❌ Not necessarily.

A primary key is a **constraint**.

A clustered index is an **index structure**.

They are separate concepts.

You can have:

PRIMARY KEY NONCLUSTERED

and then have another column with a clustered index.

Example conceptually:

EmployeeId → PK + Nonclustered

CreatedAt → Clustered Index

7. Composite Index

An index can contain multiple columns.

```
CREATE INDEX IX_Employees_Department_Name  
ON Employees(DepartmentId, Name);
```

This is a:

Composite index

The order matters.

(DepartmentId, Name)

is not equivalent to:

(Name, DepartmentId)

8. Leftmost Prefix / Leading Column

Suppose:

```
CREATE INDEX IX_Employees  
ON Employees(DepartmentId, Name);
```

The **leading column** is:

DepartmentId

Queries filtering by:

```
WHERE DepartmentId = 5
```

can make good use of the index.

Also:

```
WHERE DepartmentId = 5  
AND Name = 'Ahmed'
```

can use the composite key effectively.

But:

```
WHERE Name = 'Ahmed'
```

doesn't get the same benefit from this index because Name isn't the leading key.

Interview answer:

In a composite index, column order matters; the leading key columns are especially important for seekability.

9. Included Columns

Suppose:

```
SELECT Name, Salary
FROM Employees
WHERE DepartmentId = 5;
```

You could create:

```
CREATE INDEX IX_Employees_Department
ON Employees(DepartmentId)
INCLUDE (Name, Salary);
```

Here:

DepartmentId

is an index **key**.

While:

Name
Salary

are **included columns**.

The purpose is often to let the index provide all required columns for the query.

10. Covering Index

An index that contains everything needed by a query is called a **covering index** for that query.

Example:

```
CREATE INDEX IX_Employees_Department
ON Employees(DepartmentId)
INCLUDE (Name, Salary);
```

Query:

```
SELECT Name, Salary
FROM Employees
WHERE DepartmentId = 5;
```

The index contains:

```
WHERE → DepartmentId
SELECT → Name, Salary
```

So SQL Server may be able to retrieve everything from the index without going back to the base table.

This can avoid a costly lookup.

11. Index Seek

An **Index Seek** means SQL Server can navigate into an index to find the relevant portion of data.

Conceptually:

```
Index
↓
DepartmentId = 5
↓
directly locate relevant rows
```

Usually preferable when you're retrieving a relatively selective subset.

12. Index Scan

An **Index Scan** means SQL Server scans a larger portion or all of the index.

Conceptually:

```
Index
↓
row
row
row
row
```

row

...

A scan isn't automatically bad.

For example:

```
SELECT COUNT(*)  
FROM Employees;
```

If almost every row is needed, scanning may be cheaper than repeatedly seeking.

Important interview point:

Seek = not always good, Scan = not always bad.

The optimizer chooses based on estimated cost.

13. Table Scan

A table scan reads through the table's data pages.

If the table is a heap (no clustered index), you may see a:

Table Scan

If the table has a clustered index, scanning the entire clustered index is commonly shown as:

Clustered Index Scan

14. Key Lookup

Very important.

Suppose you have:

```
CREATE INDEX IX_Employees_Department  
ON Employees(DepartmentId);
```

Query:

```
SELECT Name, Salary
FROM Employees
WHERE DepartmentId = 5;
```

The index finds the matching employees, but doesn't contain:

Name
Salary

SQL Server may perform a:

Key Lookup

to retrieve those columns from the clustered index/base table.

Conceptually:

Nonclustered Index Seek

↓
matching rows
↓
Key Lookup
↓
actual data

A covering index can sometimes eliminate that lookup.

15. Execution Plan

An execution plan shows **how SQL Server intends to execute a query** and, when viewing an actual execution plan, what actually happened.

Example:

```
SELECT *
FROM Employees
WHERE DepartmentId = 5;
```

You might see:

Index Seek
↓
Key Lookup

or:

Clustered Index Scan

The execution plan helps you understand:

- Which indexes were used
 - Whether SQL Server scanned or sought
 - Join algorithms
 - Sorts
 - Lookups
 - Estimated/actual rows
 - Expensive operators
-

16. Estimated vs Actual Execution Plan

Estimated plan

SQL Server generates the plan **without executing the query**.

Useful for asking:

“What does SQL Server think it will do?”

Actual plan

SQL Server executes the query and gives you runtime information.

Useful for:

“What actually happened?”

In SSMS:

Ctrl + L

→ Estimated execution plan

Ctrl + M

→ Include actual execution plan

17. Execution Plan Operators

You should recognize these:

Index Seek

Find specific index entries

Index Scan

Scan much/all of an index

Table Scan

Scan heap table

Key Lookup

Go back to clustered/base data for missing columns

Sort

Sort rows

Nested Loops

Good in many cases when one input is small and the other has an efficient lookup.

Hash Match

Often useful for larger, unsorted inputs.

Merge Join

Works particularly well when both inputs are already ordered appropriately.

18. SARGability

🔥 Very important interview concept.

SARGable means the query predicate is written in a way that allows SQL Server to efficiently use an index, often through a seek.

Bad:

```
WHERE YEAR(OrderDate) = 2026
```

You applied a function to the indexed column.

Better:

```
WHERE OrderDate >= '20260101'  
AND OrderDate < '20270101'
```

The second form gives SQL Server a searchable range on OrderDate.

Another example:



```
WHERE Name LIKE '%Ahmed'
```

The leading wildcard makes normal B-tree index seeking difficult.

Potentially better:

```
WHERE Name LIKE 'Ahmed%'
```

because the search has a known prefix.

19. Functions on Indexed Columns

Another common problem:

```
WHERE LOWER(Name) = 'ahmed'
```

or:

```
WHERE YEAR(OrderDate) = 2026
```

The function changes the expression SQL Server needs to search.

Instead, try to express the condition as a range or otherwise in a form that preserves index usability.

Interview phrase:

“Avoid applying functions to indexed columns in predicates when possible because it can make the predicate non-SARGable.”

20. Statistics

SQL Server's optimizer uses **statistics** to estimate things like:

How many rows will this predicate return?

For example:

WHERE DepartmentId = 5

If SQL Server estimates:

10 rows

it may choose an index seek.

If it estimates:

900,000 rows

a scan may be cheaper.

So:

Statistics

↓

Cardinality estimation

↓

Optimizer

↓

Execution plan

This is why outdated or inaccurate statistics can contribute to poor plans.

21. Selectivity

Selectivity describes how well a predicate narrows down rows.

Suppose 1,000,000 employees exist.

WHERE Id = 500

probably returns one row.

Very selective.

But:

```
WHERE IsActive = 1
```

might return 950,000 rows.

Low selectivity.

A highly selective predicate is often a better candidate for an index seek.

22. Composite Index Example

Suppose your common query is:

```
SELECT *  
FROM Orders  
WHERE CustomerId = 10  
AND OrderDate >= '20260101';
```

Potential index:

```
CREATE INDEX IX_Orders_Customer_Date  
ON Orders(CustomerId, OrderDate);
```

Why?

Because your search pattern begins with:

```
CustomerId
```

and then narrows by:

```
OrderDate
```

The exact optimal index depends on workload, selectivity, included columns, ordering, and actual execution plans.

★ Interview Questions

Q1. What is an index?

A database structure that helps SQL Server locate data more efficiently.

Q2. Why not create indexes on every column?

Indexes consume storage and increase the cost of INSERT, UPDATE, and DELETE operations.

Q3. Clustered vs nonclustered?

A clustered index determines the organization of the table's data by its key and only one can exist per table. A nonclustered index is a separate structure and multiple can exist.

Q4. Can a table have multiple clustered indexes?

No. Only one.

Q5. Can a table have multiple nonclustered indexes?

Yes.

Q6. Is a primary key always clustered?

No. A primary key is a constraint; its index can be clustered or nonclustered.

Q7. What is a composite index?

An index containing multiple key columns.

Q8. Does column order matter in a composite index?

Yes. The leading columns strongly affect which predicates can efficiently use the index.

Q9. What is a covering index?

An index that contains all columns needed by a query, allowing SQL Server to satisfy the query from the index without needing additional lookups.

Q10. Seek vs Scan?

Seek navigates to relevant index entries; scan reads a larger portion or all of an index/table.

Q11. Is a scan always bad?

No. If a large percentage of the table is needed, scanning may be cheaper than seeking.

Q12. What is SARGability?

Writing predicates in a form that allows the optimizer to efficiently use indexes, often through seeks.



The Interview Cheat Sheet

INDEX

- faster reads
- extra storage
- slower writes

CLUSTERED

- one per table
- organizes table data by key

NONCLUSTERED

- multiple allowed
- separate structure

COMPOSITE

- multiple key columns
- column order matters

INCLUDE

- extra non-key columns
- useful for covering queries

COVERING INDEX

- contains everything query needs

SEEK

- targeted navigation

SCAN

- reads large/all portion

KEY LOOKUP

- fetch missing columns from base/clustered data

STATISTICS

→ help optimizer estimate rows

SARGABLE

→ index-friendly predicate

★ The most important mental model:

Query



Statistics



Query Optimizer



Execution Plan



Seek / Scan / Join / Sort / Lookup



Result

Next: Group 13 — Triggers & Cursors. We'll cover what they are, AFTER vs INSTEAD OF triggers, inserted/deleted tables, recursion, and why cursors are usually avoided.

go

Group 13 — Triggers & Cursors

This is a **very common SQL Server interview group**, especially for backend/.NET roles.

1. What is a Trigger?

A **trigger** is a special database object that automatically executes when a specified event occurs.

For DML triggers, the events are:

- INSERT
- UPDATE

- DELETE

Unlike a stored procedure, **you don't normally call a trigger manually**; the database fires it automatically. ([Microsoft Learn](#))

Example idea

Suppose you have:

Customers

Id

Name

Balance

and:

CustomerAudit

Id

CustomerId

Action

ActionDate

Whenever a customer is deleted, you could have a trigger automatically insert an audit record.

2. Trigger vs Stored Procedure

Trigger	Stored Procedure
Automatically executes	Explicitly executed
Associated with table/view or event	Independent database object
Fired by INSERT/UPDATE/DELETE etc.	Called with EXEC
Common for auditing/enforcement	Common for reusable business operations
Can affect performance unexpectedly	More explicit/control flow

Interview question

Can I call a trigger using EXEC?

✗ No. A trigger fires because its associated event occurs.

3. DML Trigger Types

The two important types are:

AFTER Trigger

Runs **after** the triggering DML operation has successfully passed the relevant constraint/referential processing.

AFTER INSERT

AFTER UPDATE

AFTER DELETE

FOR is essentially an alternative spelling for AFTER in this context. ([Microsoft Learn](#))

INSTEAD OF Trigger

Runs **instead of** the original operation.

For example:

INSTEAD OF DELETE

means:

Don't perform the normal DELETE; execute my trigger logic instead.

INSTEAD OF triggers are particularly useful with **views**. ([Microsoft Learn](#))

Key difference

AFTER

Client → DELETE → actual DELETE → trigger

INSTEAD OF

Client → DELETE → trigger → trigger decides what happens

4. inserted and deleted Tables ★★

This is **extremely important for interviews**.

SQL Server provides two special temporary tables inside DML triggers:

- inserted
- deleted

They contain the rows affected by the operation. ([Microsoft Learn](#))

INSERT

inserted → new rows

deleted → empty

DELETE

inserted → empty

deleted → deleted/old rows

UPDATE

deleted → old values

inserted → new values

Think:

UPDATE

Before → deleted

After → inserted

SQL Server internally treats an update conceptually like deleting the old version and inserting the new version for purposes of these trigger tables. ([Microsoft Learn](#))

5. Example: Audit UPDATE

Suppose:

Employees

Id
Salary

An employee changes:

Salary: 10000 → 12000

Inside an UPDATE trigger:

deleted

Id = 1
Salary = 10000

inserted

Id = 1
Salary = 12000

So you can compare:

deleted.Salary
inserted.Salary

and record:

OldSalary = 10000
NewSalary = 12000

6. VERY IMPORTANT: Trigger Is NOT Per Row

This is a classic interview trap.

Suppose:

```
DELETE FROM Employees
WHERE DepartmentId = 5;
```

This might delete **500 rows**.

The trigger fires **once for the statement**, and deleted can contain all 500 rows.

It does **not** execute once per row like:

```
row 1 → trigger
row 2 → trigger
row 3 → trigger
...
```

Instead:

```
DELETE statement
    ↓
  trigger
    ↓
deleted = 500 rows
```

Therefore, trigger logic must be designed to handle **multiple rows**. ([Microsoft Learn](#))

✗ Bad mindset

"inserted contains one row"

✓ Correct mindset

"inserted/deleted are rowsets"

This is one of the most important trigger interview points.

7. Why Are Triggers Useful?

Common uses:

1. Auditing

```
Employee changed
    ↓
  Trigger
```

↓
Audit table

2. Enforcing complex business rules

Especially rules involving multiple tables that aren't conveniently expressed with a simple constraint. ([Microsoft Learn](#))

3. Automatically maintaining related information

For example:

OrderItem inserted

↓

Trigger

↓

Update order total

But you should be careful with this approach.

8. Problems with Triggers ★

Triggers can be dangerous if overused.

1. Hidden behavior

You execute:

INSERT INTO Orders ...

but another table changes because a trigger secretly executes.

2. Performance

A poorly designed trigger can make an ordinary INSERT/UPDATE/DELETE expensive.

3. Complexity

Debugging becomes harder:

API

↓

SQL UPDATE

↓
Trigger
↓
Another UPDATE
↓
Another trigger
↓
...

4. Multi-row mistakes

A trigger written assuming one row may produce incorrect results when the statement affects many rows.

Microsoft specifically recommends rowset-based logic rather than cursors inside triggers. ([Microsoft Learn](#))

9. Trigger + Transaction ★

A DML trigger and the statement that fired it are treated as part of the same transaction.

So if the trigger causes the transaction to roll back, the original operation can also be rolled back. ([Microsoft Learn](#))

Conceptually:

```
BEGIN
  INSERT
  ↓
  Trigger
  ↓
  Something fails
  ↓
  ROLLBACK
END
```

Therefore, triggers can prevent a modification from successfully completing.

10. DDL Triggers

Don't confuse these with DML triggers.

DML

INSERT
UPDATE
DELETE

DDL

CREATE
ALTER
DROP
GRANT
DENY
REVOKE
...

DDL triggers respond to database/schema-level events such as CREATE, ALTER, and DROP. They don't use the inserted/deleted tables. ([Microsoft Learn](#))

For most junior interviews, know the distinction; DML triggers are more important.

11. Disable vs Drop Trigger

Disable

Trigger still exists
↓
but doesn't fire

Drop

Trigger object is removed

A disabled trigger can later be enabled again. ([Microsoft Learn](#))

12. What is a Cursor?

A **cursor** allows you to process query results **row by row**.

Normal SQL encourages:

Think in sets.

Cursor:

Give me one row at a time.

Conceptually:

Query result

↓

Row 1 → process

Row 2 → process

Row 3 → process

...

13. Cursor Lifecycle

Remember this sequence:

DECLARE

↓

OPEN

↓

FETCH

↓

FETCH

↓

...

CLOSE

↓

DEALLOCATE

Meaning

DECLARE

Define the cursor.

OPEN

Execute the cursor's query and make it ready.

FETCH

Retrieve the next row.

CLOSE

Stop using the cursor result.

DEALLOCATE

Release the cursor resources.

14. Why Are Cursors Usually Avoided?

Because SQL Server is designed to work efficiently with **sets**.

Compare:

Cursor

1 row
→ process

1 row
→ process

1 row
→ process

...

Set-based

```
UPDATE Employees  
SET Salary = Salary * 1.10  
WHERE DepartmentId = 5;
```

One statement can operate on the entire set.

Generally:

Set-based



better performance + simpler SQL

whereas:

Cursor



row-by-row processing



often slower

Microsoft also specifically recommends avoiding cursors in multi-row trigger logic because of their potential performance impact. ([Microsoft Learn](#))

15. When Might You Actually Need a Cursor?

They're not completely useless.

They can be appropriate when you genuinely need **sequential, row-by-row processing** where a set-based solution isn't practical.

But in an interview, if asked:

“Would you use a cursor?”

A strong answer is:

Only when necessary. I would first look for a set-based solution because cursors introduce row-by-row processing and can hurt performance.

Interview Questions

Q1. What is a trigger?

A database object that automatically executes in response to a specified database event.

Q2. Trigger vs stored procedure?

Trigger is automatically fired; stored procedure is explicitly called.

Q3. What are inserted and deleted?

Special trigger tables containing affected row versions.

Q4. What does inserted contain during UPDATE?

The **new values**.

Q5. What does deleted contain during UPDATE?

The **old values**.

Q6. Does a trigger execute once per row?

 No. In SQL Server, a DML trigger fires per **statement**, so it must handle multiple affected rows.

Q7. AFTER vs INSTEAD OF?

AFTER

- operation happens
- trigger runs

INSTEAD OF

- trigger runs instead of operation

Q8. Can a trigger rollback the operation?

 Yes, because the trigger and triggering DML statement participate in the same transaction.

Q9. Why avoid triggers?

Hidden behavior, debugging complexity, performance overhead, and difficult maintenance.

Q10. What is a cursor?

A mechanism for processing a result set row by row.

Q11. Cursor lifecycle?

DECLARE → OPEN → FETCH → CLOSE → DEALLOCATE

Q12. Cursor vs set-based SQL?

Cursor → row by row

Set-based → whole set

Prefer **set-based SQL** whenever practical.

Group 13 Cheat Sheet

TRIGGER

→ automatically executes

DML TRIGGER

→ INSERT / UPDATE / DELETE

AFTER

→ executes after the operation

INSTEAD OF

→ executes instead of the operation

inserted

→ new rows

deleted

→ old/deleted rows

UPDATE

→ deleted = old

→ inserted = new

IMPORTANT

→ Trigger fires per statement, NOT per row

DDL TRIGGER

→ CREATE / ALTER / DROP etc.

CURSOR

→ row-by-row processing

CURSOR LIFECYCLE

- DECLARE
- OPEN
- FETCH
- CLOSE
- DEALLOCATE

PREFERRED

→ Set-based SQL > Cursor

Next group: SQL Server-specific advanced topics — temporary tables vs table variables, IDENTITY, SEQUENCE, OUTPUT, MERGE, PIVOT/UNPIVOT, and useful system functions.

go

Group 14 — SQL Server Advanced / Practical Features

This group is high-yield for a .NET + **SQL Server interview**. Focus especially on **temp tables vs table variables, IDENTITY vs SEQUENCE, OUTPUT, and PIVOT**.

1. Temporary Tables #Temp

A temporary table is a real table created in tempdb and available during your session/scope.

```
CREATE TABLE #Employees
(
    Id INT,
    Name VARCHAR(100)
);
```

```
INSERT INTO #Employees
VALUES (1, 'Ahmed'), (2, 'Ali');
```

```
SELECT * FROM #Employees;
```

Local temporary tables start with:

```
#Employees
```

They are normally available to the current session and automatically removed when their scope ends.

Why use them?

Useful when you need to:

- store intermediate results
 - perform multiple operations on intermediate data
 - index the intermediate result
 - work with relatively larger temporary datasets
-

2. Table Variables @table

A table variable is declared using DECLARE.

```
DECLARE @Employees TABLE
(
    Id INT,
    Name VARCHAR(100)
);
```

```
INSERT INTO @Employees
VALUES (1, 'Ahmed'), (2, 'Ali');
```

```
SELECT * FROM @Employees;
```

Notice:

```
#Employees → temporary table
@Employees → table variable
```

3. Temp Table vs Table Variable ★★ ★

	#Temp	@Table
Syntax	CREATE TABLE #T	DECLARE @T TABLE
Stored in	tempdb	tempdb internally
Can have indexes	Yes	More limited/indexing

	#Temp	@Table
		through keys/index definitions
Statistics	Generally has statistics	Historically more limited; modern SQL Server has improved behavior
Good for	Intermediate datasets / more complex work	Smaller/simple temporary result sets
Can be reused within scope	Yes	Yes

Interview answer

Don't say:

“Table variables are always faster.”

That's incorrect.

A better answer:

The choice depends on the workload, row count, indexing, and query plan. Temp tables are generally more suitable when the intermediate result is larger or needs richer indexing/statistics behavior.

4. IDENTITY ★★ ★

IDENTITY automatically generates numeric values for a column.

```
CREATE TABLE Employees
(
    Id INT IDENTITY(1,1),
    Name VARCHAR(100)
);
```

Meaning:

IDENTITY(seed, increment)

IDENTITY(1,1)

produces:

1
2
3
4
...

Another example:

IDENTITY(100,10)

produces:

100
110
120
130
...

Important

IDENTITY does **not guarantee gapless numbers**.

For example:

1
2
3
5
6

4 might be missing because of a rollback, deletion, failed operation, etc.

So don't use IDENTITY when you need:

“Every number must be consecutive with no gaps.”

5. IDENTITY vs Primary Key

These are completely different concepts.

IDENTITY

Answers:

How is the value generated?

PRIMARY KEY

Answers:

How is the row uniquely identified?

You can have:

```
Id INT IDENTITY PRIMARY KEY
```

which combines both.

But they are conceptually different.

6. SEQUENCE

A SEQUENCE is a database object that generates numbers.

```
CREATE SEQUENCE OrderSequence  
AS INT  
START WITH 1  
INCREMENT BY 1;
```

Then:

```
SELECT NEXT VALUE FOR OrderSequence;
```

returns the next number.

Main difference

IDENTITY:

belongs to a column/table

SEQUENCE:

independent database object

Therefore, a sequence can be used by **multiple tables**.

Interview question

When would you choose SEQUENCE over IDENTITY?

When you need more control over number generation or want the same sequence to be used across multiple tables/processes.

7. OUTPUT Clause

OUTPUT lets you retrieve rows affected by:

INSERT
UPDATE
DELETE
MERGE

SQL Server exposes values such as:

INSERTED
DELETED

through the OUTPUT clause. ([Microsoft Learn](#))

Example: Get Generated ID

Instead of:

INSERT
↓
SELECT MAX(Id)

which is unsafe in concurrent scenarios,

you can use:

```
INSERT INTO Employees(Name)
OUTPUT INSERTED.Id
VALUES ('Ahmed');
```

You get the newly generated ID directly.

This is **very useful in backend development**.

8. OUTPUT INSERTED VS OUTPUT DELETED

Think of it similarly to triggers:

INSERT

INSERTED → new row

DELETE

DELETED → deleted row

UPDATE

DELETED → old values

INSERTED → new values

Example:

```
UPDATE Employees
SET Salary = 15000
OUTPUT
    DELETED.Salary AS OldSalary,
    INSERTED.Salary AS NewSalary
WHERE Id = 1;
```

Result:

OldSalary	NewSalary
12000	15000

9. OUTPUT INTO

You can also store the output into a table or table variable. ([Microsoft Learn](#))

Conceptually:

```
UPDATE
↓
OUTPUT
↓
@TableVariable
```

Useful for capturing affected IDs or changes for further processing.

10. MERGE

MERGE can combine different operations depending on whether a source row matches the target.

Conceptually:

```
Source
↓
Does target row exist?
↓
┌───┴───┐
|         |
YES       NO
|         |
UPDATE    INSERT
```

Example concept:

```
MERGE Customers AS Target
USING CustomerImport AS Source
ON Target.Id = Source.Id
```

```
WHEN MATCHED THEN
  UPDATE SET Target.Name = Source.Name
```

```
WHEN NOT MATCHED BY TARGET THEN
```

```
INSERT (Id, Name)
VALUES (Source.Id, Source.Name);
```

SQL Server supports MERGE with actions such as UPDATE, INSERT, and DELETE, and its OUTPUT clause can report which action occurred. ([Microsoft Learn](#))

Interview caution

You should **not blindly say MERGE is always the best way to implement upsert**. In real production systems, teams often prefer separate UPDATE/INSERT logic depending on concurrency, correctness requirements, and known SQL Server MERGE edge cases.

For a junior interview:

MERGE is used when you need to synchronize a target with a source based on matching conditions.

That's enough.

11. PIVOT ★★

PIVOT transforms **rows into columns**, usually with aggregation. ([Microsoft Learn](#))

Suppose:

Employee	Year	Sales
Ahmed	2024	100
Ahmed	2025	150
Ali	2024	200
Ali	2025	250

You can pivot into:

Employee	2024	2025
Ahmed	100	150
Ali	200	250

Think:

PIVOT
rows → columns

12. UNPIVOT

The opposite general direction:

columns → rows

Example:

Employee	2024	2025
Ahmed	100	150

becomes something like:

Employee	Year	Sales
Ahmed	2024	100
Ahmed	2025	150

UNPIVOT rotates columns into rows. ([Microsoft Learn](#))

Important nuance

UNPIVOT isn't necessarily a perfect reversible operation because PIVOT can aggregate multiple source rows together. ([Microsoft Learn](#))

13. PIVOT vs Conditional Aggregation

You can often accomplish the same type of result with:

SUM(CASE WHEN Year = 2024 THEN Sales ELSE 0 END)

instead of PIVOT.

For interviews, know both approaches.

14. ISNULL VS COALESCE

Very common SQL interview question.

ISNULL

SQL Server-specific:

```
ISNULL(Phone, 'No Phone')
```

COALESCE

Standard SQL expression:

```
COALESCE(Phone, Email, 'No Contact')
```

It returns the first non-NULL expression.

Example:

Phone	Email
NULL	a@gmail.com

```
COALESCE(Phone, Email)
```

returns:

```
a@gmail.com
```

Key difference

ISNULL

- SQL Server-specific
- two arguments

COALESCE

- standard SQL
 - can have multiple arguments
-

15. CASE

Conditional logic inside SQL.

```
SELECT
  Name,
  Salary,
  CASE
    WHEN Salary >= 20000 THEN 'High'
    WHEN Salary >= 10000 THEN 'Medium'
    ELSE 'Low'
  END AS SalaryLevel
FROM Employees;
```

Think:

if
else if
else

but inside SQL.

Interview Cheat Sheet

#Temp

- temporary table
- useful for intermediate/complex datasets

@Table

- table variable
- useful for temporary/simple results

IDENTITY

- automatic value generation for a column
- does NOT guarantee gapless numbers

SEQUENCE

- independent number generator
- can be shared by multiple tables

OUTPUT

- returns affected rows/values
- INSERTED / DELETED
- useful for retrieving generated IDs

MERGE

- synchronize source and target
- MATCHED / NOT MATCHED

PIVOT

- rows → columns

UNPIVOT

- columns → rows

ISNULL

- SQL Server-specific
- 2 arguments

COALESCE

- standard SQL
- multiple arguments

CASE

- conditional logic

★ Most likely interview questions from this group

1. **Temp table vs table variable?**
2. **IDENTITY vs SEQUENCE?**
3. **Does IDENTITY guarantee consecutive numbers?** → ❌
4. **How do you get the generated ID after INSERT?** → OUTPUT INSERTED.Id
5. **What does OUTPUT DELETED mean in UPDATE?** → old values
6. **What does PIVOT do?** → rows → columns
7. **What does UNPIVOT do?** → columns → rows
8. **ISNULL vs COALESCE?**
9. **What is MERGE used for?**
10. **Can a sequence be shared by multiple tables?** → ✅

Next: Group 15 — Advanced SQL interview problems: duplicate removal, gaps & islands, running totals, top-N-per-group, consecutive dates, latest record per group, and real interview-style queries.